

Ingeniería de Software II

Patrones de diseño

Universidad Autónoma de Entre Ríos
Facultad de Ciencia y Tecnología

Lic. Paolo Orundés Cardinali
orundescardinali.paolo@uader.edu.ar

Lic. Damián Cian
cian.damian@uader.edu.ar

Sumérgete en los patrones de diseño

Alexander Shvets, Refactoring.Guru, 2019

support@refactoring.guru

<https://refactoring.guru/es/design-patterns>

¿Qué es un patrón de diseño?

Los patrones de diseño (design patterns) son soluciones habituales a problemas que ocurren con frecuencia en el diseño de software. Son como planos prefabricados que se pueden personalizar para resolver un problema de diseño recurrente en tu código.

No se puede elegir un patrón y copiarlo en el programa como si se tratara de funciones o bibliotecas ya preparadas. El patrón no es una porción específica de código, sino un concepto general para resolver un problema particular. Puedes seguir los detalles del patrón e implementar una solución que encaje con las realidades de tu propio programa.

A menudo los patrones se confunden con algoritmos porque ambos conceptos describen soluciones típicas a problemas conocidos. Mientras que un algoritmo siempre define un grupo claro de acciones para lograr un objetivo, un patrón es una descripción de más alto nivel de una solución. El código del mismo patrón aplicado a dos programas distintos puede ser diferente.

Una analogía de un algoritmo sería una receta de cocina: ambos cuentan con pasos claros para alcanzar una meta. Por su parte, un patrón es más similar a un plano, ya que puedes observar cómo son su resultado y sus funciones, pero el orden exacto de la implementación depende de ti.

¿En qué consiste el patrón?

La mayoría de los patrones se describe con mucha formalidad para que la gente pueda reproducirlos en muchos contextos. Aquí tienes las secciones que suelen estar presentes en la descripción de un patrón:

El propósito del patrón explica brevemente el problema y la solución.

La motivación explica en más detalle el problema y la solución que brinda el patrón.

La estructura de las clases muestra cada una de las partes del patrón y el modo en que se relacionan.

El ejemplo de código en uno de los lenguajes de programación populares facilita la asimilación de la idea que se esconde tras el patrón.

En síntesis: Los patrones de diseño de software son soluciones probadas y efectivas para problemas comunes en el diseño de software. Estos patrones facilitan la creación de sistemas más flexibles, mantenibles y escalables.

Historia de los patrones

¿Quién inventó los patrones de diseño? Esa es una buena, aunque imprecisa pregunta. Los patrones de diseño no son conceptos opacos y sofisticados, al contrario. Los patrones son soluciones habituales a problemas comunes en el diseño orientado a objetos. Cuando una solución se repite una y otra vez en varios proyectos, al final alguien le pone un nombre y explica la solución en detalle. Básicamente, así es como se descubre un patrón.

El concepto de los patrones fue descrito por **Christopher Alexander en El lenguaje de patrones**. El libro habla de un “lenguaje” para diseñar el entorno urbano. Las unidades de este lenguaje son los patrones. Pueden describir lo altas

que tienen que ser las ventanas, cuántos niveles debe tener un edificio, cuan grandes deben ser las zonas verdes de un barrio, etcétera.

La idea fue recogida por cuatro autores: Erich Gamma, John Vlissides, Ralph Johnson y Richard Helm. En 1995, publicaron Patrones de diseño, en el que aplicaron el concepto de los patrones de diseño a la programación. El libro presentaba 23 patrones que resolvían varios problemas del diseño orientado a objetos y se convirtió en un éxito de ventas con rapidez. Al tener un título tan largo en inglés, la gente empezó a llamarlo “el libro de la ‘gang of four’ (banda de los cuatro)”, lo que pronto se abrevió a “el libro GoF”.

Desde entonces se han descubierto decenas de nuevos patrones orientados a objetos. La “metodología del patrón” se hizo muy popular en otros campos de la programación, por lo que hoy en día existen muchos otros patrones no relacionados con el diseño orientado a objetos.

Ventaja de los patrones

¿Por qué debería aprender sobre patrones?

La realidad es que podrías trabajar durante años como programador sin conocer un solo patrón. Mucha gente lo hace. Incluso en ese caso, podrías estar implementando patrones sin saberlo. Así que, ¿por qué dedicar tiempo a aprenderlos?

Los patrones de diseño son un juego de herramientas de soluciones comprobadas a problemas habituales en el diseño de software. Incluso aunque nunca te encuentres con estos problemas, conocer los patrones sigue siendo de utilidad, porque te enseña a resolver todo tipo de problemas utilizando principios del diseño orientado a objetos.

Los patrones de diseño definen un lenguaje común que puedes utilizar con tus compañeros de equipo para comunicaros de forma más eficiente. Podrías decir: “Oh, utiliza un singleton para eso”, y todos entenderían la idea de tu sugerencia. No habría necesidad de explicar qué es un singleton si conocen el patrón y su nombre.

Crítica de los patrones

Da la sensación de que todos los holgazanes han criticado ya los patrones de diseño. Veamos los argumentos más habituales contra el uso de los patrones.

Chapuzas para un lenguaje de programación débil

Normalmente, la necesidad por los patrones surge cuando la gente elige un lenguaje de programación o una tecnología que carece del nivel necesario de abstracción. En este caso, los patrones se convierten en una chapuza que otorga al lenguaje unas súper habilidades muy necesitadas.

Por ejemplo, el patrón Strategy puede implementarse con una simple función anónima (lambda) en la mayoría de lenguajes de programación modernos.

Soluciones ineficientes

Los patrones intentan sistematizar soluciones cuyo uso ya es generalizado. Esta unificación es vista por muchos como un dogma, e implementan los patrones “al pie de la letra”, sin adaptarlos al contexto del proyecto particular.

Uso injustificado

“Si lo único que tienes es un martillo, todo te parecerá un clavo.”

Este es el problema que persigue a muchos principiantes que acaban de familiarizarse con los patrones. Una vez que aprenden sobre patrones, intentan aplicarlos en todas partes, incluso en situaciones en las que un código más simple funcionaría perfectamente bien.

Clasificación de patrones

Los patrones de diseño varían en su complejidad, nivel de detalle y escala de aplicabilidad al sistema completo que se diseña. Una analogía que usa el autor es la de la construcción de carreteras: puedes hacer más segura una intersección instalando semáforos o construyendo un intercambiador completo de varios niveles con pasajes subterráneos para peatones.

Por su nivel

Los patrones más básicos y de más bajo nivel suelen llamarse idioms. Normalmente se aplican a un único lenguaje de programación.

Los patrones más universales y de más alto nivel son los patrones de arquitectura. Los desarrolladores pueden implementar estos patrones prácticamente en cualquier lenguaje. Al contrario que otros patrones, pueden utilizarse para diseñar la arquitectura de una aplicación completa.

Por su propósito

Patrones creacionales

Proporcionan mecanismos de creación de objetos que incrementan la flexibilidad y la reutilización de código existente.

Patrones estructurales

Explican cómo ensamblar objetos y clases en estructuras más grandes a la vez que se mantiene la flexibilidad y eficiencia de la estructura.

Patrones de comportamiento

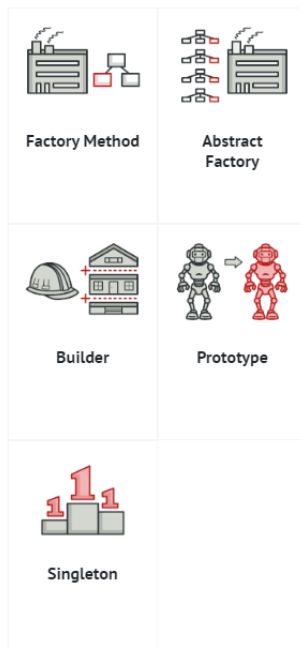
Se encargan de una comunicación efectiva y la asignación de responsabilidades entre objetos.



El catálogo de patrones de diseño

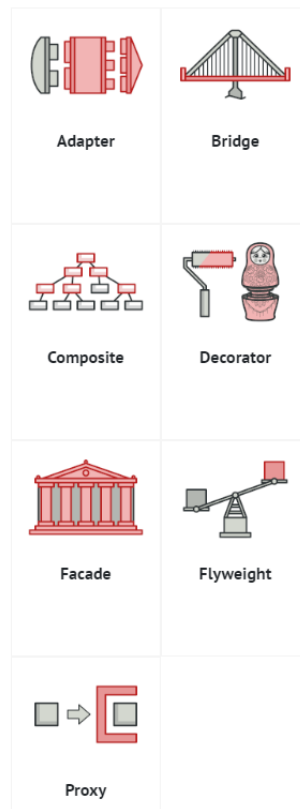
Patrones creacionales

Estos patrones proporcionan mecanismos de creación de objetos que incrementan la flexibilidad y la reutilización del código existente.



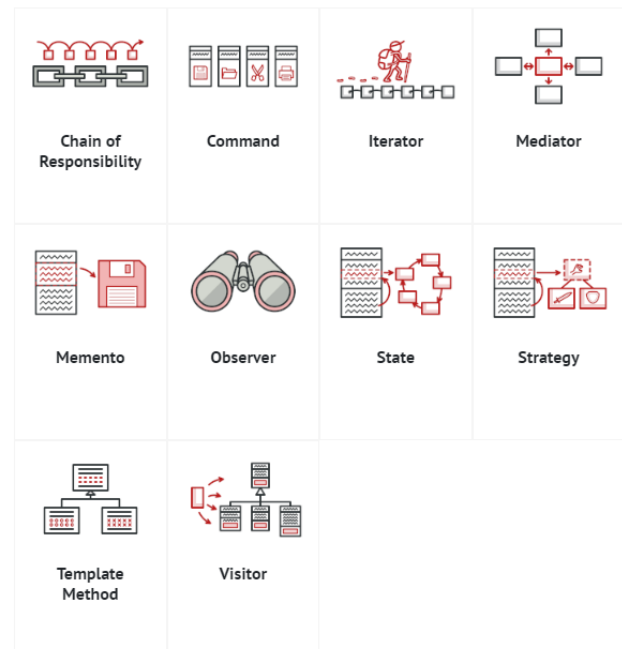
Patrones estructurales

Estos patrones explican cómo ensamblar objetos y clases en estructuras más grandes, mientras se mantiene la flexibilidad y eficiencia de la estructura.



Patrones de comportamiento

Estos patrones tratan con algoritmos y la asignación de responsabilidades entre objetos.



<https://refactoring.guru/es/design-patterns/catalog>

1. Patrones creacionales:

- **Singleton:** Garantiza que una clase solo tenga una única instancia y proporciona un punto de acceso global a ella.
- **Factory Method:** Permite a una clase delegar la creación de objetos a sus subclases, sin especificar la clase concreta.
- **Abstract Factory:** Proporciona una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas.
- **Builder:** Abstrae la construcción de un objeto complejo, permitiendo crear diferentes representaciones del mismo proceso de construcción.
- **Prototype:** nos permite copiar objetos existentes sin que el código dependa de sus clases.

2. Patrones estructurales:

- **Adapter:** Permite que clases incompatibles trabajen juntas mediante la conversión de interfaces.
- **Decorator:** Agrega comportamiento adicional a objetos dinámicamente sin modificar su estructura.
- **Proxy:** Actúa como intermediario para controlar el acceso a un objeto y añadir funcionalidades adicionales.
- **Composite:** Permite tratar objetos individuales y composiciones de objetos de manera uniforme, tratándolos como una estructura jerárquica.

3. Patrones de comportamiento:

- **Observer:** Establece una dependencia uno-a-muchos entre objetos, de modo que cuando un objeto cambia de estado, todos los objetos dependientes son notificados y actualizados automáticamente.
- **Strategy:** Permite cambiar dinámicamente el algoritmo utilizado por un objeto en tiempo de ejecución.
- **Command:** Encapsula una solicitud como un objeto, lo que permite parametrizar clientes con diferentes solicitudes y soportar operaciones reversibles (deshacer).
- **Template Method:** Define el esqueleto de un algoritmo en una clase base, permitiendo que las subclasses implementen ciertos pasos del algoritmo sin cambiar su estructura general.

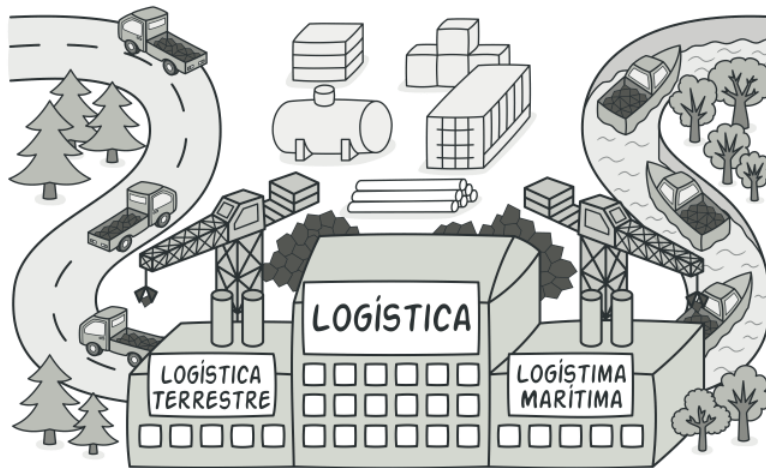
PATRONES CREACIONALES

Factory Method

Propósito

Factory Method es un patrón de diseño creacional que proporciona una interfaz para crear objetos en una superclase, mientras permite a las subclasses alterar el tipo de objetos que se crearán.

<https://refactoring.guru/es/design-patterns/factory-method>



Problema

Imagina que estás creando una aplicación de gestión logística. La primera versión de tu aplicación sólo es capaz de manejar el transporte en camión, por lo que la mayor parte de tu código se encuentra dentro de la clase **Camión**.

Al cabo de un tiempo, tu aplicación se vuelve bastante popular. Cada día recibes decenas de peticiones de empresas de transporte marítimo para que incorpores la logística por mar a la aplicación.



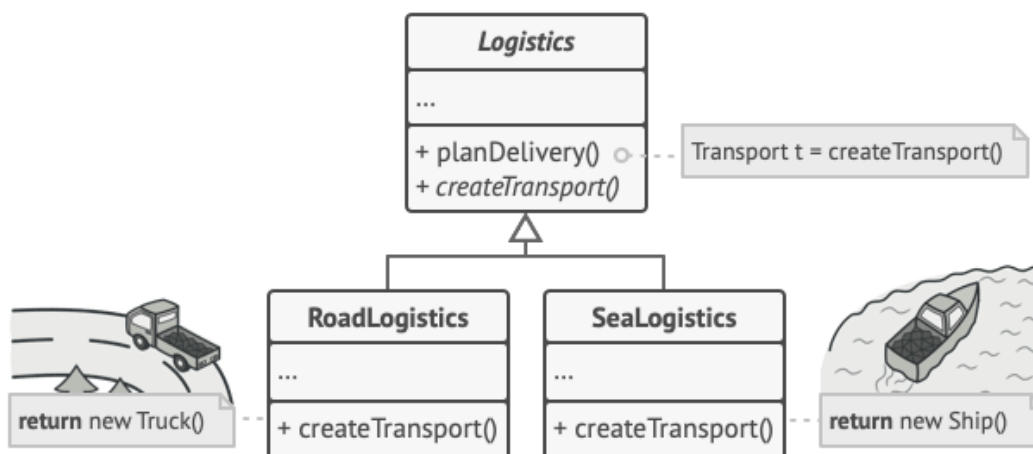
Añadir una nueva clase al programa no es tan sencillo si el resto del código ya está acoplado a clases existentes.

Estupendo, ¿verdad? Pero, ¿qué pasa con el código? En este momento, la mayor parte de tu código está acoplado a la clase Camión. Para añadir barcos a la aplicación habría que hacer cambios en toda la base del código. Además, si más tarde decides añadir otro tipo de transporte a la aplicación, probablemente tendrás que volver a hacer todos estos cambios.

Al final acabarás con un código bastante sucio, plagado de condicionales que cambian el comportamiento de la aplicación dependiendo de la clase de los objetos de transporte.

Solución

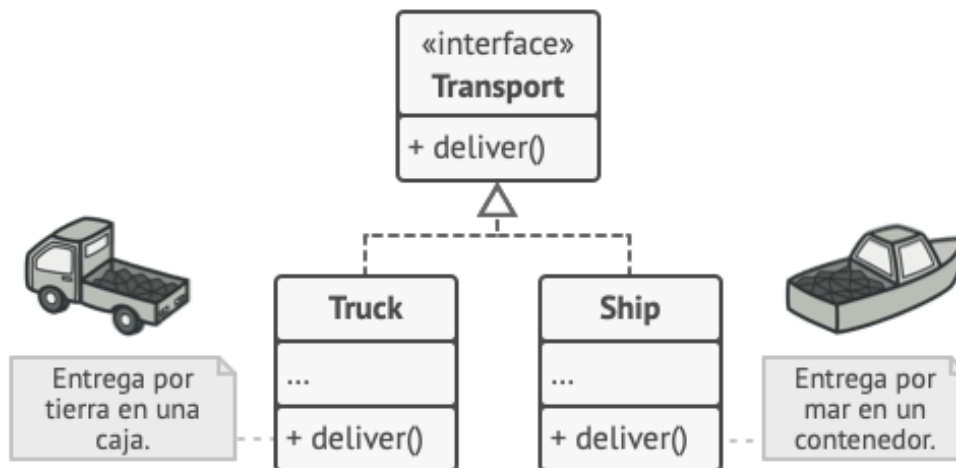
El patrón Factory Method sugiere que, en lugar de llamar al operador **new** para construir objetos directamente, se invoque a un método fábrica especial. No te preocupes: los objetos se siguen creando a través del operador new, pero se invocan desde el método fábrica. Los objetos devueltos por el método fábrica a menudo se denominan productos.



Las subclases pueden alterar la clase de los objetos devueltos por el método fábrica.

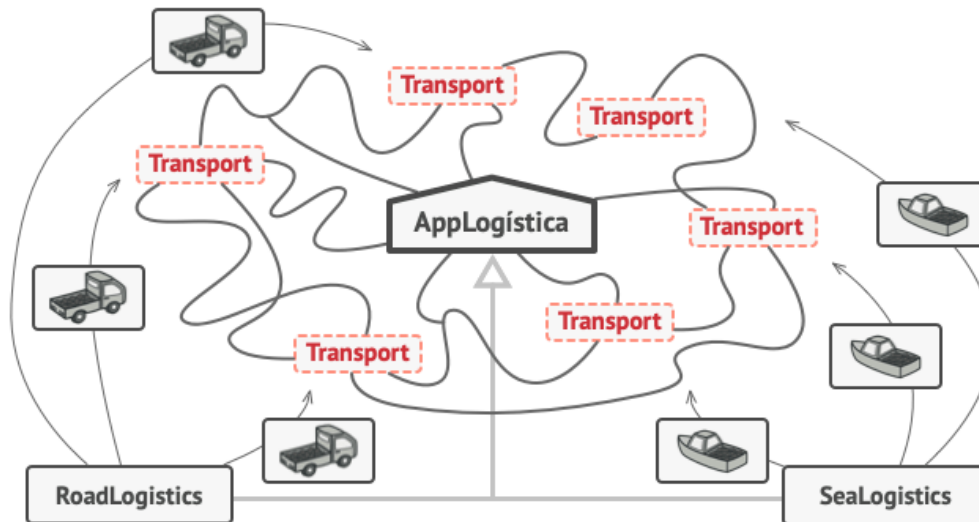
A simple vista, puede parecer que este cambio no tiene sentido, ya que tan solo hemos cambiado el lugar desde donde invocamos al constructor. Sin embargo, piensa en esto: ahora puedes sobrescribir el método fábrica en una subclase y cambiar la clase de los productos creados por el método.

No obstante, hay una pequeña limitación: las subclases sólo pueden devolver productos de distintos tipos si dichos productos tienen una clase base o interfaz común. Además, el método fábrica en la clase base debe tener su tipo de retorno declarado como dicha interfaz.



Todos los productos deben seguir la misma interfaz.

Por ejemplo, tanto la clase **Camión** como la clase **Barco** deben implementar la interfaz Transporte, que declara un método llamado entrega. Cada clase implementa este método de forma diferente: los camiones entregan su carga por tierra, mientras que los barcos lo hacen por mar. El método fábrica dentro de la clase **LogísticaTerrestre** devuelve objetos de tipo camión, mientras que el método fábrica de la clase **LogísticaMarítima** devuelve barcos.



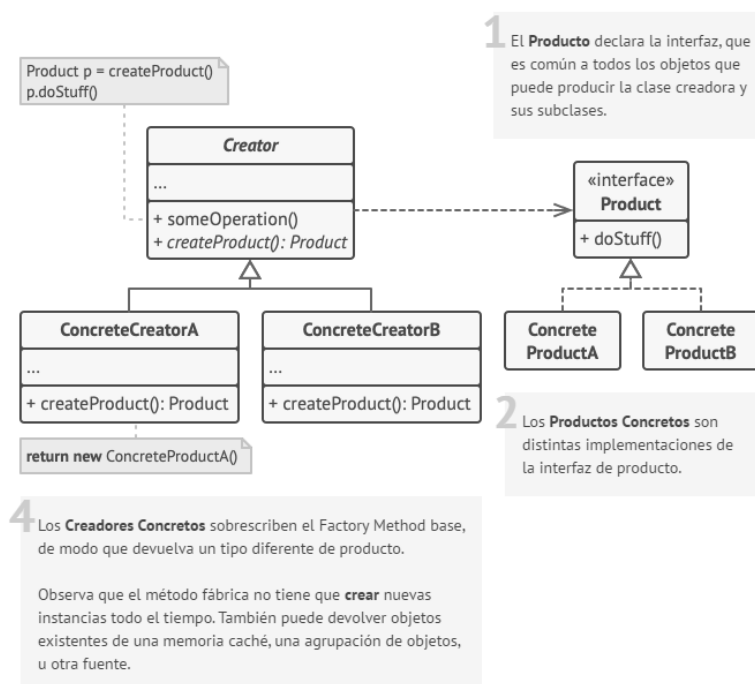
Siempre y cuando todas las clases de producto implementen una interfaz común, podrás pasar sus objetos al código cliente sin descomponerlo.

El código que utiliza el método fábrica (a menudo denominado código cliente) no encuentra diferencias entre los productos devueltos por varias subclases, y trata a todos los productos como la clase abstracta **Transporte**. El cliente sabe que todos los objetos de transporte deben tener el método **entrega**, pero no necesita saber cómo funciona exactamente.

Estructura

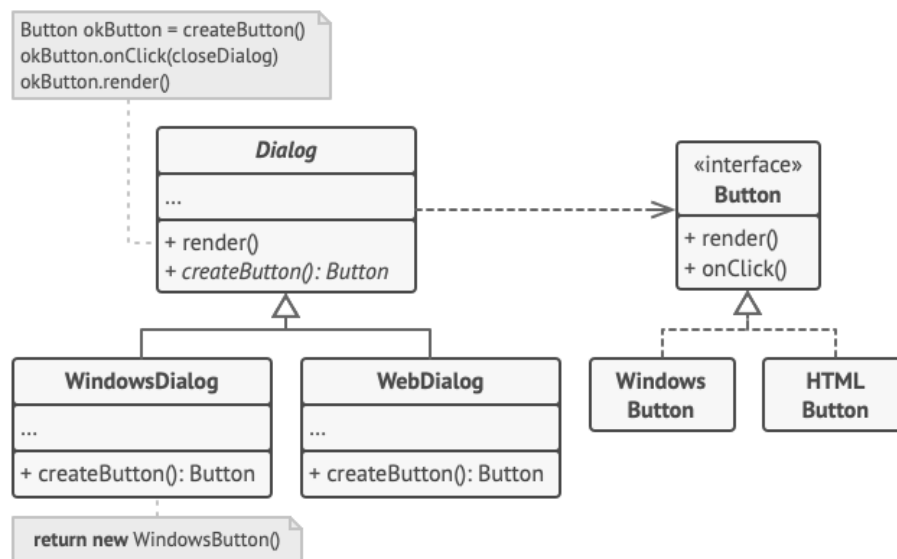
Estructura

- 3** La clase **Creadora** declara el método fábrica que devuelve nuevos objetos de producto. Es importante que el tipo de retorno de este método coincida con la interfaz de producto.
- Puedes declarar el patrón Factory Method como abstracto para forzar a todas las subclases a implementar sus propias versiones del método. Como alternativa, el método fábrica base puede devolver algún tipo de producto por defecto.
- Observa que, a pesar de su nombre, la creación de producto no es la principal responsabilidad de la clase creadora. Normalmente, esta clase cuenta con alguna lógica de negocios central relacionada con los productos. El patrón Factory Method ayuda a desacoplar esta lógica de las clases concretas de producto. Aquí tienes una analogía: una gran empresa de desarrollo de software puede contar con un departamento de formación de programadores. Sin embargo, la principal función de la empresa sigue siendo escribir código, no preparar programadores.



Pseudocódigo

Este ejemplo ilustra cómo puede utilizarse el patrón **Factory Method** para crear elementos de interfaz de usuario (UI) multiplataforma sin acoplar el código cliente a clases UI concretas.



Ejemplo del diálogo multiplataforma.

La clase base de diálogo utiliza distintos elementos UI para representar su ventana. En varios sistemas operativos, estos elementos pueden tener aspectos diferentes, pero su comportamiento debe ser consistente. Un botón en Windows sigue siendo un botón en Linux.

Cuando entra en juego el patrón Factory Method no hace falta reescribir la lógica del diálogo para cada sistema operativo. Si declaramos un patrón Factory Method que produce botones dentro de la clase base de diálogo, más tarde podremos crear una subclase de diálogo que devuelva botones al estilo de Windows desde el Factory Method. Entonces la subclase hereda la mayor parte del código del diálogo de la clase base, pero, gracias al Factory Method, puede representar botones al estilo de Windows en pantalla.

Para que este patrón funcione, la clase base de diálogo debe funcionar con botones abstractos, es decir, una clase base o una interfaz que sigan todos los botones concretos. De este modo, el código sigue siendo funcional, independientemente del tipo de botones con el que trabaje.

Por supuesto, también se puede aplicar este sistema a otros elementos UI. Sin embargo, con cada nuevo método de fábrica que añadas al diálogo, más te acercará al patrón **Abstract Factory**. No temas, más adelante hablaremos sobre este patrón.

```

// La clase creadora declara el método fábrica que debe devolver
// un objeto de una clase de producto. Normalmente, las
// subclases de la creadora proporcionan la implementación de
// este método.
class Dialog is
    // La creadora también puede proporcionar cierta
    // implementación por defecto del método fábrica.
    abstract method createButton():Button

    // Observa que, a pesar de su nombre, la principal
    // responsabilidad de la creadora no es crear productos.
    // Normalmente contiene cierta lógica de negocio que depende
    // de los objetos de producto devueltos por el método
    // fábrica. Las subclases pueden cambiar indirectamente esa
    // lógica de negocio sobrescribiendo el método fábrica y
    // devolviendo desde él un tipo diferente de producto.
    method render() is
        // Invoca el método fábrica para crear un objeto de
        // producto.
        Button okButton = createButton()
        // Ahora utiliza el producto.
        okButton.onClick(closeDialog)
        okButton.render()

// Los creadores concretos sobrescriben el método fábrica para
// cambiar el tipo de producto resultante.
class WindowsDialog extends Dialog is
    method createButton():Button is
        return new WindowsButton()

class WebDialog extends Dialog is
    method createButton():Button is
        return new HTMLButton()

// La interfaz de producto declara las operaciones que todos los
// productos concretos deben implementar.
interface Button is
    method render()
    method onClick(f)

// Los productos concretos proporcionan varias implementaciones
// de la interfaz de producto.

class WindowsButton implements Button is
    method render(a, b) is
        // Representa un botón en estilo Windows.
    method onClick(f) is
        // Vincula un evento clic de OS nativo.

class HTMLButton implements Button is
    method render(a, b) is
        // Devuelve una representación HTML de un botón.
    method onClick(f) is
        // Vincula un evento clic de navegador web.

class Application is
    field dialog: Dialog

    // La aplicación elige un tipo de creador dependiendo de la
    // configuración actual o los ajustes del entorno.
    method initialize() is
        config = readApplicationConfigFile()

        if (config.OS == "Windows") then
            dialog = new WindowsDialog()
        else if (config.OS == "Web") then
            dialog = new WebDialog()
        else
            throw new Exception("Error! Unknown operating system.")

    // El código cliente funciona con una instancia de un
    // creador concreto, aunque a través de su interfaz base.
    // Siempre y cuando el cliente siga funcionando con el
    // creador a través de la interfaz base, puedes pasarle
    // cualquier subclase del creador.
    method main() is
        this.initialize()
        dialog.render()

```

Aplicabilidad

“Utiliza el Método Fábrica cuando no conozcas de antemano las dependencias y los tipos exactos de los objetos con los que deba funcionar tu código.”

- ✓ El patrón Factory Method separa el código de construcción de producto del código que hace uso del producto. Por ello, es más fácil extender el código de construcción de producto de forma independiente al resto del código.

Por ejemplo, para añadir un nuevo tipo de producto a la aplicación, sólo tendrás que crear una nueva subclase creadora y sobrescribir el Factory Method que contiene.

“Utiliza el Factory Method cuando quieras ofrecer a los usuarios de tu biblioteca o framework, una forma de extender sus componentes internos.”

- ✓ La herencia es probablemente la forma más sencilla de extender el comportamiento por defecto de una biblioteca o un framework. Pero, ¿cómo reconoce el framework si debe utilizar tu subclase en lugar de un componente estándar?

La solución es reducir el código que construye componentes en todo el framework a un único patrón Factory Method y permitir que cualquiera sobrescriba este método además de extender el propio componente.

Veamos cómo funcionaría. Imagina que escribes una aplicación utilizando un framework de UI de código abierto. Tu aplicación debe tener botones redondos, pero el framework sólo proporciona botones cuadrados. Extiendes la clase estándar Botón con una maravillosa subclase BotónRedondo, pero ahora tienes que decirle a la clase principal FrameworkUI que utilice la nueva subclase de botón en lugar de la clase por defecto.

Para conseguirlo, creamos una subclase UIConBotonesRedondos a partir de una clase base del framework y sobrescribimos su método crearBotón. Si bien este método devuelve objetos Botón en la clase base, haces que tu subclase devuelva objetos BotónRedondo. Ahora, utiliza la clase UIConBotonesRedondos en lugar de FrameworkUI. ¡Eso es todo!

“Utiliza el Factory Method cuando quieras ahorrar recursos del sistema mediante la reutilización de objetos existentes en lugar de reconstruirlos cada vez.”

- ✓ A menudo experimentas esta necesidad cuando trabajas con objetos grandes y que consumen muchos recursos, como conexiones de bases de datos, sistemas de archivos y recursos de red.

Pensemos en lo que hay que hacer para reutilizar un objeto existente:

1. Primero, debemos crear un almacenamiento para llevar un registro de todos los objetos creados.
2. Cuando alguien necesite un objeto, el programa deberá buscar un objeto disponible dentro de ese agrupamiento.
3. ... y devolverlo al código cliente.
4. Si no hay objetos disponibles, el programa deberá crear uno nuevo (y añadirlo al agrupamiento).

¡Eso es mucho código! Y hay que ponerlo todo en un mismo sitio para no contaminar el programa con código duplicado.

Es probable que el lugar más evidente y cómodo para colocar este código sea el constructor de la clase cuyos objetos intentamos reutilizar. Sin embargo, un constructor siempre debe devolver nuevos objetos por definición. No puede devolver instancias existentes.

Por lo tanto, necesitas un método regular capaz de crear nuevos objetos, además de reutilizar los existentes. Eso suena bastante a lo que hace un patrón Factory Method.

Implementación

1. Haz que todos los productos sigan la misma interfaz. Esta interfaz deberá declarar métodos que tengan sentido en todos los productos.
2. Añade un patrón Factory Method vacío dentro de la clase creadora. El tipo de retorno del método deberá coincidir con la interfaz común de los productos.
3. Encuentra todas las referencias a constructores de producto en el código de la clase creadora. Una a una, sustitúyelas por invocaciones al Factory Method, mientras extraes el código de creación de productos para colocarlo dentro del Factory Method.

Puede ser que tengas que añadir un parámetro temporal al Factory Method para controlar el tipo de producto devuelto.

A estas alturas, es posible que el aspecto del código del Factory Method luzca bastante desagradable. Puede ser que tenga un operador switch largo que elige qué clase de producto instanciar. Pero, no te preocupes, lo arreglaremos enseguida.

4. Ahora, crea un grupo de subclases creadoras para cada tipo de producto enumerado en el Factory Method. Sobrescribe el Factory Method en las subclases y extrae las partes adecuadas de código constructor del método base.
5. Si hay demasiados tipos de producto y no tiene sentido crear subclases para todos ellos, puedes reutilizar el parámetro de control de la clase base en las subclases.

Por ejemplo, imagina que tienes la siguiente jerarquía de clases: la clase base Correo con las subclases CorreoAéreo y CorreoTerrestre y la clase Transporte con Avión, Camión y Tren. La clase CorreoAéreo sólo utiliza objetos Avión, pero CorreoTerrestre puede funcionar tanto con objetos Camión, como con objetos Tren. Puedes crear una nueva subclase (digamos, CorreoFerroviario) que gestione ambos casos, pero hay otra opción. El código cliente puede pasar un argumento al Factory Method de la clase CorreoTerrestre para controlar el producto que quiere recibir.

6. Si, tras todas las extracciones, el Factory Method base queda vacío, puedes hacerlo abstracto. Si queda algo dentro, puedes convertirlo en un comportamiento por defecto del método.

Pros y contras

- ✓ Evitas un acoplamiento fuerte entre el creador y los productos concretos.
- ✓ Principio de responsabilidad única. Puedes mover el código de creación de producto a un lugar del programa, haciendo que el código sea más fácil de mantener.
- ✓ Principio de abierto/cerrado. Puedes incorporar nuevos tipos de productos en el programa sin descomponer el código cliente existente.
- ✗ Puede ser que el código se complique, ya que debes incorporar una multitud de nuevas subclases para implementar el patrón. La situación ideal sería introducir el patrón en una jerarquía existente de clases creadoras.

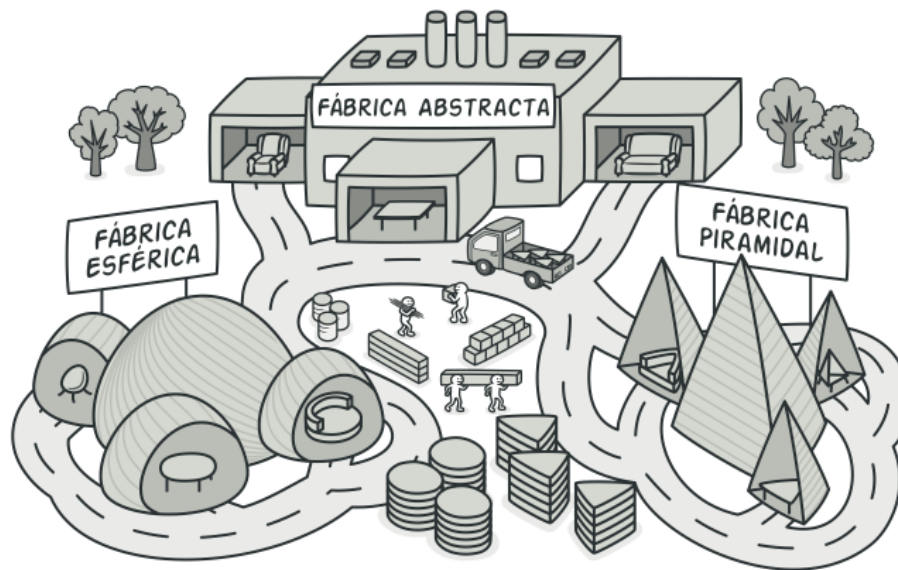
Relaciones con otros patrones

- Muchos diseños empiezan utilizando el Factory Method (menos complicado y más personalizable mediante las subclases) y evolucionan hacia Abstract Factory, Prototype, o Builder (más flexibles, pero más complicados).
- Las clases del Abstract Factory a menudo se basan en un grupo de métodos de fábrica, pero también puedes utilizar Prototype para escribir los métodos de estas clases.
- Puedes utilizar el patrón Factory Method junto con el Iterator para permitir que las subclases de la colección devuelvan distintos tipos de iteradores que sean compatibles con las colecciones.
- Prototype no se basa en la herencia, por lo que no presenta sus inconvenientes. No obstante, Prototype requiere de una inicialización complicada del objeto clonado. Factory Method se basa en la herencia, pero no requiere de un paso de inicialización.
- Factory Method es una especialización del Template Method. Al mismo tiempo, un Factory Method puede servir como paso de un gran Template Method.

Abstract Factory

Propósito

Abstract Factory es un patrón de diseño creacional que nos permite producir familias de objetos relacionados sin especificar sus clases concretas.



Problema

Imagina que estás creando un simulador de tienda de muebles. Tu código está compuesto por clases que representan lo siguiente:

1. Una familia de productos relacionados, digamos: **Silla** + **Sofá** + **Mesilla**.
2. Algunas variantes de esta familia. Por ejemplo, los productos **Silla** + **Sofá** + **Mesilla** están disponibles en estas variantes: **Moderna**, **Victoriana**, **ArtDecó**.

	Silla	Sofá	Mesilla
Art Decó			
Victoriana			
Moderna			

Familias de productos y sus variantes.

Necesitamos una forma de crear objetos individuales de mobiliario para que combinen con otros objetos de la misma familia. Los clientes se enfadan bastante cuando reciben muebles que no combinan.

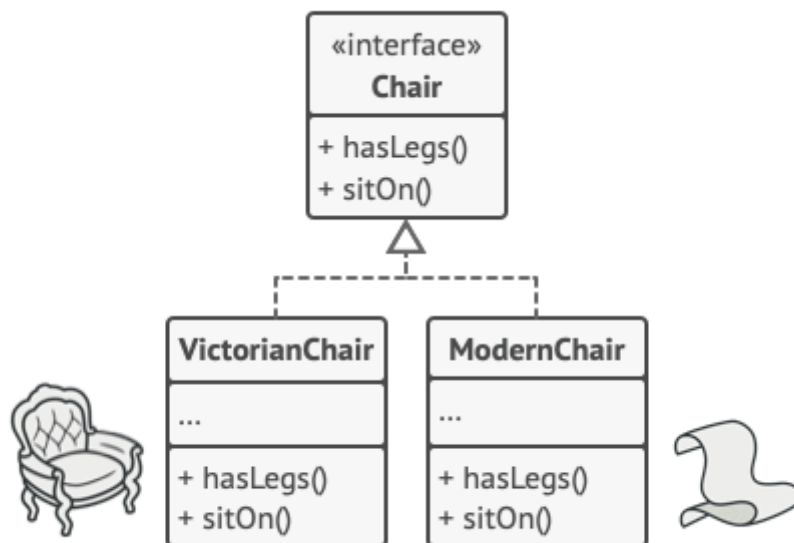


Un sofá de estilo moderno no combina con unas sillas de estilo victoriano.

Además, no queremos cambiar el código existente al añadir al programa nuevos productos o familias de productos. Los comerciantes de muebles actualizan sus catálogos muy a menudo, y debemos evitar tener que cambiar el código principal cada vez que esto ocurra.

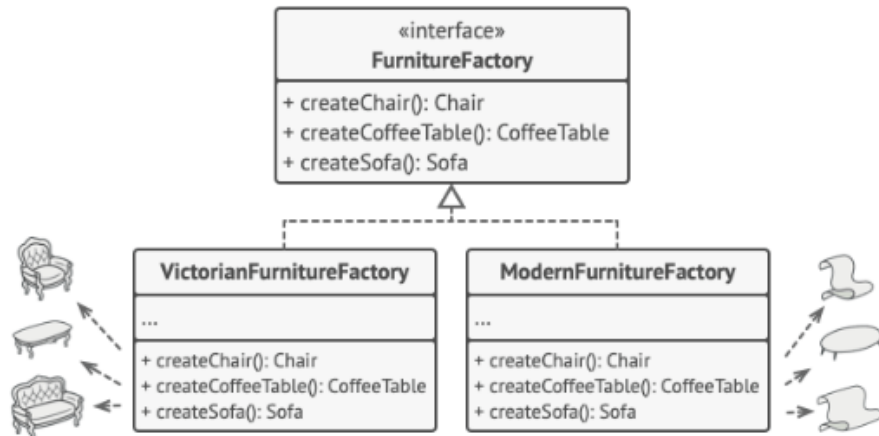
Solución

Lo primero que sugiere el patrón Abstract Factory es que declaremos de forma explícita interfaces para cada producto diferente de la familia de productos (por ejemplo, silla, sofá o mesilla). Después podemos hacer que todas las variantes de los productos sigan esas interfaces. Por ejemplo, todas las variantes de silla pueden implementar la interfaz `Silla`, así como todas las variantes de mesilla pueden implementar la interfaz `Mesilla`, y así sucesivamente.



Todas las variantes del mismo objeto deben moverse a una única jerarquía de clase.

El siguiente paso consiste en declarar la Fábrica abstracta: una interfaz con una lista de métodos de creación para todos los productos que son parte de la familia de productos (por ejemplo, `crearSilla`, `crearSofá` y `crearMesilla`). Estos métodos deben devolver productos abstractos representados por las interfaces que extrajimos previamente: `Silla`, `Sofá`, `Mesilla`, etc.



Cada fábrica concreta se corresponde con una variante específica del producto.

Ahora bien, ¿qué hay de las variantes de los productos? Para cada variante de una familia de productos, creamos una clase de fábrica independiente basada en la interfaz `FábricaAbstracta`. Una fábrica es una clase que devuelve productos de un tipo particular. Por ejemplo, la `Fábrica de Muebles Modernos` sólo puede crear objetos de `SillaModerna`, `SofáModerno` y `MesillaModerna`.

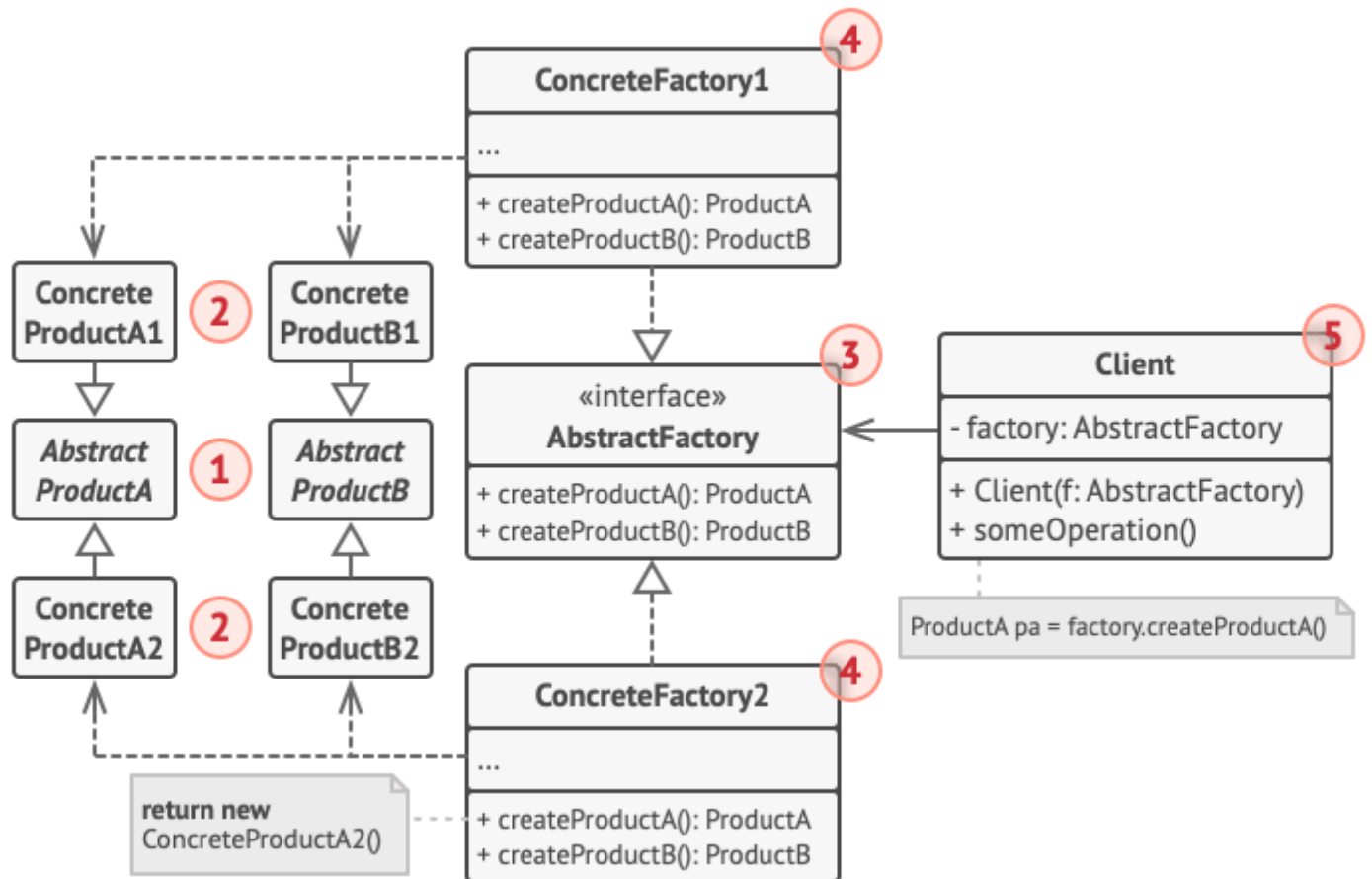
El código cliente tiene que funcionar con fábricas y productos a través de sus respectivas interfaces abstractas. Esto nos permite cambiar el tipo de fábrica que pasamos al código cliente, así como la variante del producto que recibe el código cliente, sin descomponer el propio código cliente.



Al cliente no le debe importar la clase concreta de la fábrica con la que funciona.

Digamos que el cliente quiere una fábrica para producir una silla. El cliente no tiene que conocer la clase de la fábrica y tampoco importa el tipo de silla que obtiene. Ya sea un modelo moderno o una silla de estilo victoriano, el cliente debe tratar a todas las sillas del mismo modo, utilizando la interfaz abstracta `Silla`. Con este sistema, lo único que sabe el cliente sobre la silla es que implementa de algún modo el método `sentarse`. Además, sea cual sea la variante de silla devuelta, siempre combinará con el tipo de sofá o mesilla producida por el mismo objeto de fábrica.

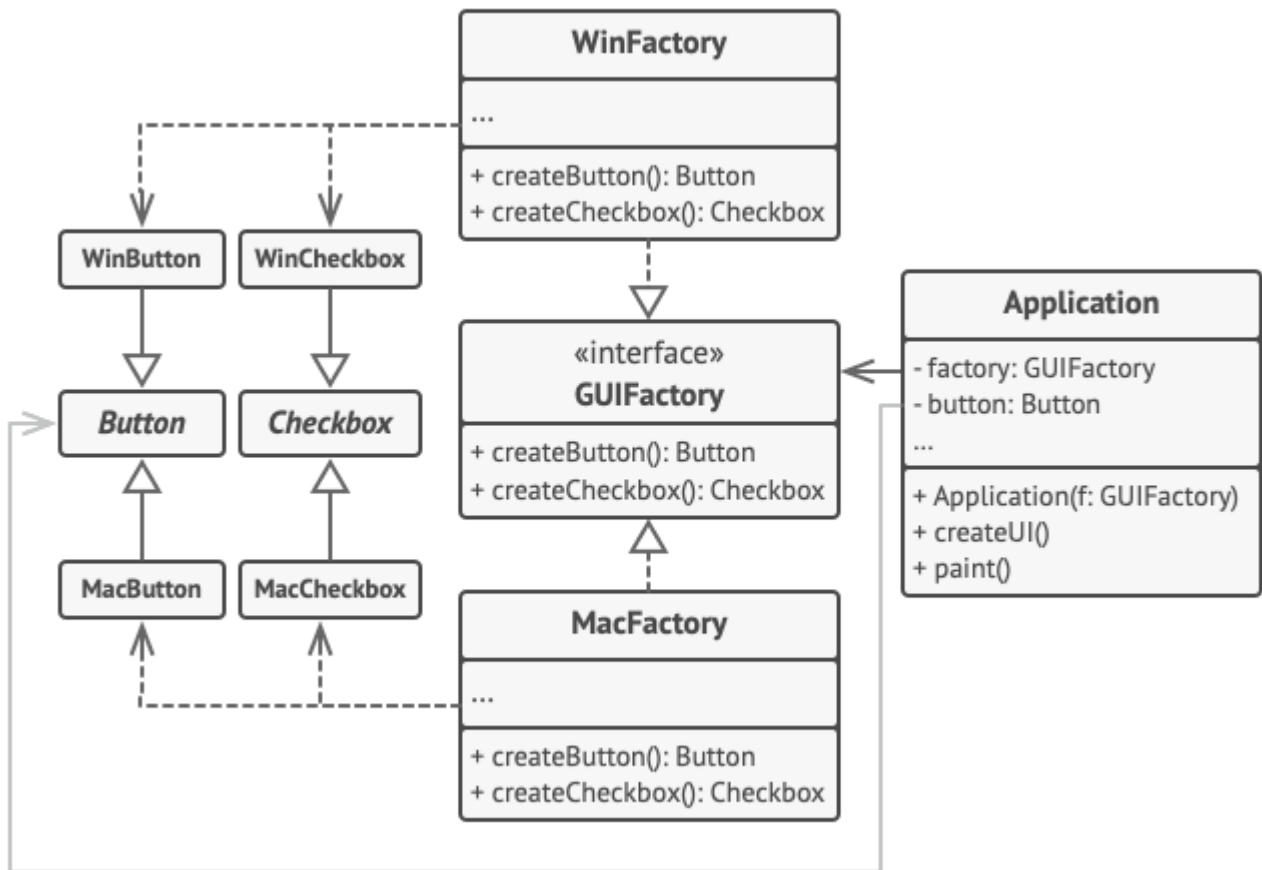
Queda otro punto por aclarar: si el cliente sólo está expuesto a las interfaces abstractas, ¿cómo se crean los objetos de fábrica? Normalmente, la aplicación crea un objeto de fábrica concreto en la etapa de inicialización. Justo antes, la aplicación debe seleccionar el tipo de fábrica, dependiendo de la configuración o de los ajustes del entorno.



1. Los **Productos Abstractos** declaran interfaces para un grupo de productos diferentes pero relacionados que forman una familia de productos.
2. Los **Productos Concretos** son implementaciones distintas de productos abstractos agrupados por variantes. Cada producto abstracto (silla/sofá) debe implementarse en todas las variantes dadas (victoriano/moderno).
3. La interfaz **Fábrica Abstracta** declara un grupo de métodos para crear cada uno de los productos abstractos.
4. Las **Fábricas Concretas** implementan métodos de creación de la fábrica abstracta. Cada fábrica concreta se corresponde con una variante específica de los productos y crea tan solo dichas variantes de los productos.
5. Aunque las fábricas concretas instancian productos concretos, las firmas de sus métodos de creación deben devolver los productos *abstractos* correspondientes. De este modo, el código cliente que utiliza una fábrica no se acopla a la variante específica del producto que obtiene de una fábrica. El **Cliente** puede funcionar con cualquier variante fábrica/producto concreta, siempre y cuando se comunique con sus objetos a través de interfaces abstractas.

Pseudocódigo

Este ejemplo ilustra cómo puede utilizarse el patrón **Abstract Factory** para crear elementos de interfaz de usuario (UI) multiplataforma sin acoplar el código cliente a clases UI concretas, mientras se mantiene la consistencia de todos los elementos creados respecto al sistema operativo seleccionado.



Ejemplo de clases UI multiplataforma.

Es de esperar que los mismos elementos UI de una aplicación multiplataforma se comporten de forma parecida, aunque tengan un aspecto un poco diferente en distintos sistemas operativos. Además, es nuestro trabajo que los elementos UI coincidan con el estilo del sistema operativo en cuestión. No queremos que nuestro programa represente controles de macOS al ejecutarse en Windows.

La interfaz fábrica abstracta declara un grupo de métodos de creación que el código cliente puede utilizar para producir distintos tipos de elementos UI. Las fábricas concretas coinciden con sistemas operativos específicos y crean los elementos UI correspondientes.

Funciona así: cuando se lanza, la aplicación comprueba el tipo de sistema operativo actual. La aplicación utiliza esta información para crear un objeto de fábrica a partir de una clase que coincida con el sistema operativo. El resto del código utiliza esta fábrica para crear elementos UI. Esto evita que se creen elementos equivocados.

Con este sistema, el código cliente no depende de clases concretas de fábricas y elementos UI, siempre y cuando trabaje con estos objetos a través de sus interfaces abstractas. Esto también permite que el código cliente soporte otras fábricas o elementos UI que pudiéramos añadir más adelante.

Como consecuencia, no necesitas modificar el código cliente cada vez que añades una nueva variedad de elementos UI a tu aplicación. Tan solo debes crear una nueva clase de fábrica que produzca estos elementos y modifique ligeramente el código de inicialización de la aplicación, de modo que seleccione esa clase cuando resulte apropiado.

```

// La interfaz fábrica abstracta declara un grupo de métodos que
// devuelven distintos productos abstractos. Estos productos se
// denominan familia y están relacionados por un tema o concepto
// de alto nivel. Normalmente, los productos de una familia
// pueden colaborar entre sí. Una familia de productos puede
// tener muchas variantes, pero los productos de una variante
// son incompatibles con los productos de otra.
interface GUIFactory is
    method createButton():Button
    method createCheckbox():Checkbox

// Las fábricas concretas producen una familia de productos que
// pertenecen a una única variante. La fábrica garantiza que los
// productos resultantes sean compatibles. Las firmas de los
// métodos de las fábricas concretas devuelven un producto
// abstracto mientras que dentro del método se instancia un
// producto concreto.
class WinFactory implements GUIFactory is
    method createButton():Button is
        return new WinButton()
    method createCheckbox():Checkbox is
        return new WinCheckbox()

// Cada fábrica concreta tiene una variante de producto
// correspondiente.
class MacFactory implements GUIFactory is
    method createButton():Button is
        return new MacButton()
    method createCheckbox():Checkbox is
        return new MacCheckbox()

// Cada producto individual de una familia de productos debe
// tener una interfaz base. Todas las variantes del producto
// deben implementar esta interfaz.
interface Button is
    method paint()

// Los productos concretos son creados por las fábricas
// concretas correspondientes.
class WinButton implements Button is
    method paint() is
        // Representa un botón en estilo Windows.

class MacButton implements Button is
    method paint() is
        // Representa un botón en estilo macOS.

// Aquí está la interfaz base de otro producto. Todos los
// productos pueden interactuar entre sí, pero sólo entre
// productos de la misma variante concreta es posible una
// interacción adecuada.
interface Checkbox is
    method paint()

class WinCheckbox implements Checkbox is
    method paint() is
        // Representa una casilla en estilo Windows.

class MacCheckbox implements Checkbox is
    method paint() is
        // Representa una casilla en estilo macOS.

// El código cliente funciona con fábricas y productos
// únicamente a través de tipos abstractos: GUIFactory, Button y
// Checkbox. Esto te permite pasar cualquier subclase fábrica o
// producto al código cliente sin descomponerlo.
class Application is
    private field factory: GUIFactory
    private field button: Button
    constructor Application(factory: GUIFactory) is
        this.factory = factory
    method createUI() is
        this.button = factory.createButton()
    method paint() is
        button.paint()

// La aplicación elige el tipo de fábrica dependiendo de la
// configuración actual o de los ajustes del entorno y la crea
// durante el tiempo de ejecución (normalmente en la etapa de
// inicialización).
class ApplicationConfigurator is
    method main() is
        config = readApplicationConfigFile()

        if (config.OS == "Windows") then
            factory = new WinFactory()
        else if (config.OS == "Mac") then
            factory = new MacFactory()
        else
            throw new Exception("Error! Unknown operating system.")

        Application app = new Application(factory)

```

Aplicabilidad

- ✓ Utiliza el patrón Abstract Factory cuando tu código deba funcionar con varias familias de productos relacionados, pero no desees que dependa de las clases concretas de esos productos, ya que puede ser que no los conozcas de antemano o sencillamente quieras permitir una futura extensibilidad.
- ✓ El patrón Abstract Factory nos ofrece una interfaz para crear objetos a partir de cada clase de la familia de productos. Mientras tu código cree objetos a través de esta interfaz, no tendrás que preocuparte por crear la variante errónea de un producto que no combine con los productos que ya ha creado tu aplicación.
- ✓ Considera la implementación del patrón Abstract Factory cuando tengas una clase con un grupo de **métodos de fábrica** que nublen su responsabilidad principal.
- ✓ En un programa bien diseñado *cada clase es responsable tan solo de una cosa*. Cuando una clase lidia con varios tipos de productos, puede ser que valga la pena extraer sus métodos de fábrica para ponerlos en una clase única de fábrica o una implementación completa del patrón Abstract Factory.

Implementación

1. Mapea una matriz de distintos tipos de productos frente a variantes de dichos productos.
2. Declara interfaces abstractas de producto para todos los tipos de productos. Después haz que todas las clases concretas de productos implementen esas interfaces.
3. Declara la interfaz de la fábrica abstracta con un grupo de métodos de creación para todos los productos abstractos.
4. Implementa un grupo de clases concretas de fábrica, una por cada variante de producto.
5. Crea un código de inicialización de la fábrica en algún punto de la aplicación. Deberá instanciar una de las clases concretas de la fábrica, dependiendo de la configuración de la aplicación o del entorno actual. Pasa este objeto de fábrica a todas las clases que construyen productos.
6. Explora el código y encuentra todas las llamadas directas a constructores de producto. Sustitúyelas por llamadas al método de creación adecuado dentro del objeto de fábrica.

Pros y contras

- ✓ Puedes tener la certeza de que los productos que obtienes de una fábrica son compatibles entre sí.
- ✓ Evitas un acoplamiento fuerte entre productos concretos y el código cliente.
- ✓ *Principio de responsabilidad única*. Puedes mover el código de creación de productos a un solo lugar, haciendo que el código sea más fácil de mantener.
- ✓ *Principio de abierto/cerrado*. Puedes introducir nuevas variantes de productos sin descomponer el código cliente existente.
- ✗ Puede ser que el código se complique más de lo que debería, ya que se introducen muchas nuevas interfaces y clases junto al patrón.

Relaciones con otros patrones

- Muchos diseños empiezan utilizando el **Factory Method** (menos complicado y más personalizable mediante las subclases) y evolucionan hacia **Abstract Factory**, **Prototype**, o **Builder** (más flexibles, pero más complicados).
- **Builder** se enfoca en construir objetos complejos, paso a paso. **Abstract Factory** se especializa en crear familias de objetos relacionados. *Abstract Factory* devuelve el producto inmediatamente, mientras que *Builder* te permite ejecutar algunos pasos adicionales de construcción antes de extraer el producto.
- Las clases del **Abstract Factory** a menudo se basan en un grupo de **métodos de fábrica**, pero también puedes utilizar **Prototype** para escribir los métodos de estas clases.
- **Abstract Factory** puede servir como alternativa a **Facade** cuando tan solo deseas esconder la forma en que se crean los objetos del subsistema a partir del código cliente.

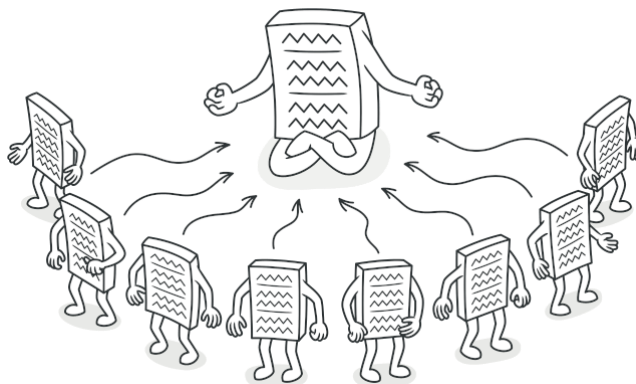
- Puedes utilizar **Abstract Factory** junto a **Bridge**. Este emparejamiento resulta útil cuando algunas abstracciones definidas por *Bridge* sólo pueden funcionar con implementaciones específicas. En este caso, *Abstract Factory* puede encapsular estas relaciones y esconder la complejidad al código cliente.
- Los patrones **Abstract Factory**, **Builder** y **Prototype** pueden todos ellos implementarse como **Singletons**.

Singleton

También llamado: Instancia única

Propósito

Singleton es un patrón de diseño creacional que nos permite asegurarnos de que una clase tenga una única instancia, a la vez que proporciona un punto de acceso global a dicha instancia.



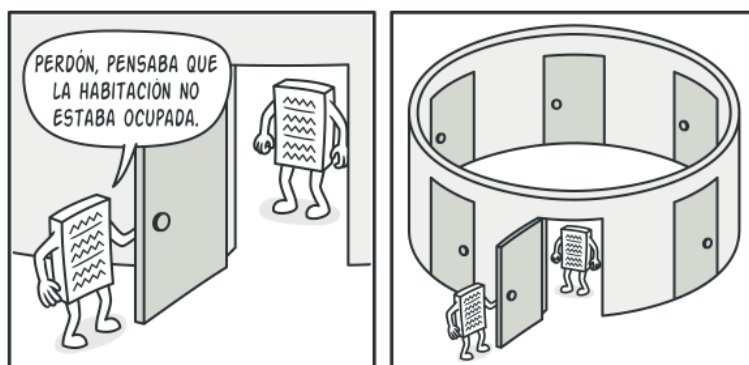
Problema

El patrón Singleton resuelve dos problemas al mismo tiempo, vulnerando el *Principio de responsabilidad única*:¹

1. **Garantizar que una clase tenga una única instancia.** ¿Por qué querría alguien controlar cuántas instancias tiene una clase? El motivo más habitual es controlar el acceso a algún recurso compartido, por ejemplo, una base de datos o un archivo.

Funciona así: imagina que has creado un objeto y al cabo de un tiempo decides crear otro nuevo. En lugar de recibir un objeto nuevo, obtendrás el que ya habías creado.

Ten en cuenta que este comportamiento es imposible de implementar con un constructor normal, ya que una llamada al constructor siempre **debe** devolver un nuevo objeto por diseño.



Puede ser que los clientes ni siquiera se den cuenta de que trabajan con el mismo objeto todo el tiempo.

2. **Proporcionar un punto de acceso global a dicha instancia.** ¿Recuerdas esas variables globales que utilizaste (bueno, sí, fui yo) para almacenar objetos esenciales? Aunque son muy útiles, también son poco seguras, ya que cualquier código podría sobrescribir el contenido de esas variables y descomponer la aplicación.

Al igual que una variable global, el patrón Singleton nos permite acceder a un objeto desde cualquier parte del programa. No obstante, también evita que otro código sobrescriba esa instancia.

¹ El principio de responsabilidad única o SRP (siglas del inglés, Single Responsibility Principle) en ingeniería de software establece que cada módulo o clase debe tener responsabilidad sobre una sola parte de la funcionalidad proporcionada por el software y esta responsabilidad debe estar encapsulada en su totalidad por la clase

Este problema tiene otra cara: no queremos que el código que resuelve el primer problema se encuentre disperso por todo el programa. Es mucho más conveniente tenerlo dentro de una clase, sobre todo si el resto del código ya depende de ella.

Hoy en día el patrón Singleton se ha popularizado tanto que la gente suele llamar *singleton* a cualquier patrón, incluso si solo resuelve uno de los problemas antes mencionados.

Solución

Todas las implementaciones del patrón Singleton tienen estos dos pasos en común:

- Hacer privado el constructor por defecto para evitar que otros objetos utilicen el operador `new` con la clase Singleton.
- Crear un método de creación estático que actúe como constructor. Tras bambalinas, este método invoca al constructor privado para crear un objeto y lo guarda en un campo estático. Las siguientes llamadas a este método devuelven el objeto almacenado en caché.

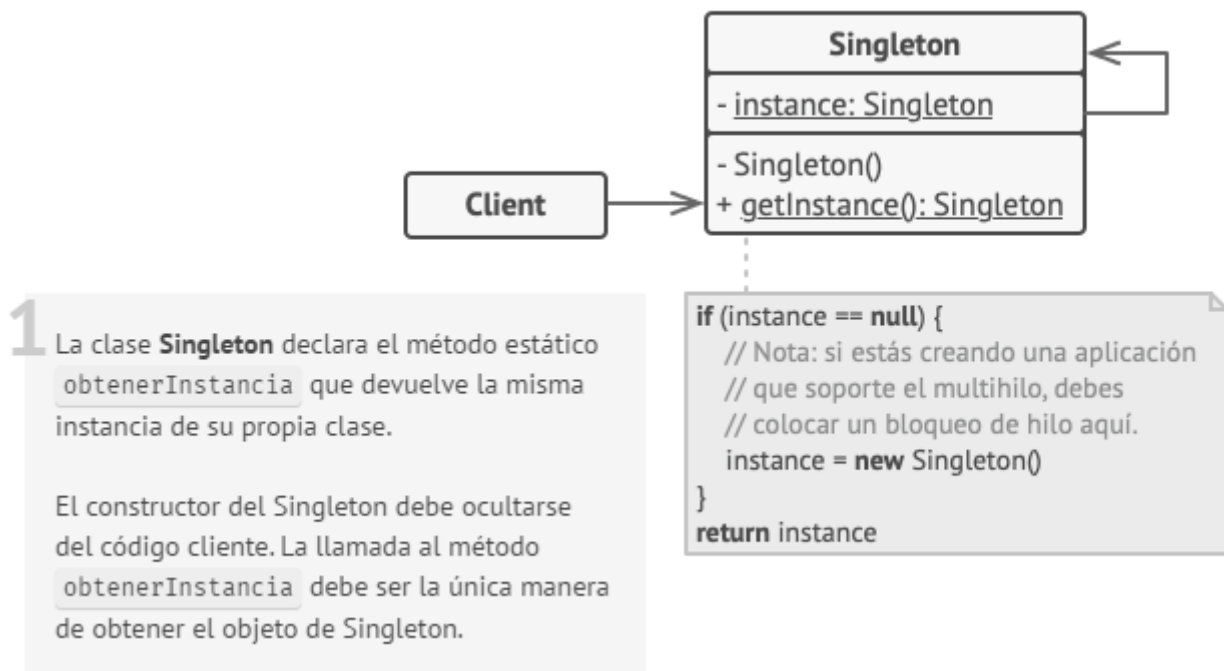
Si tu código tiene acceso a la clase Singleton, podrá invocar su método estático. De esta manera, cada vez que se invoque este método, siempre se devolverá el mismo objeto.

Analogía en el mundo real

El gobierno es un ejemplo excelente del patrón Singleton. Un país sólo puede tener un gobierno oficial.

Independientemente de las identidades personales de los individuos que forman el gobierno, el título “Gobierno de X” es un punto de acceso global que identifica al grupo de personas a cargo.

Estructura



Pseudocódigo

En este ejemplo, la clase de conexión de la base de datos actúa como **Singleton**. Esta clase no tiene un constructor público, por lo que la única manera de obtener su objeto es invocando el método `obtenerInstancia`. Este método almacena en caché el primer objeto creado y lo devuelve en todas las llamadas siguientes.

```

// La clase Base de datos define el método `obtenerInstancia`
// que permite a los clientes acceder a la misma instancia de
// una conexión de la base de datos a través del programa.
class Database is
    // El campo para almacenar la instancia singleton debe
    // declararse estático.
    private static field instance: Database

    // El constructor del singleton siempre debe ser privado
    // para evitar llamadas de construcción directas con el
    // operador `new`.
    private constructor Database() is
        // Algún código de inicialización, como la propia
        // conexión al servidor de una base de datos.
        // ...

// El método estático que controla el acceso a la instancia
// singleton.
public static method getInstance() is
    if (Database.instance == null) then
        acquireThreadLock() and then
            // Garantiza que la instancia aún no se ha

// inicializado por otro hilo mientras ésta ha
// estado esperando el desbloqueo.
        if (Database.instance == null) then
            Database.instance = new Database()
    return Database.instance

    // Por último, cualquier singleton debe definir cierta

// lógica de negocio que pueda ejecutarse en su instancia.
    public method query(sql) is

// Por ejemplo, todas las consultas a la base de datos
// de una aplicación pasan por este método. Por lo

// tanto, aquí puedes colocar lógica de regularización
// (throttling) o de envío a la memoria caché.
// ...

class Application is
    method main() is
        Database foo = Database.getInstance()
        foo.query("SELECT ...")
        // ...
        Database bar = Database.getInstance()
        bar.query("SELECT ...")
        // La variable `bar` contendrá el mismo objeto que la
        // variable `foo`.

```


Aplicabilidad

- Utiliza el patrón Singleton cuando una clase de tu programa tan solo deba tener una instancia disponible para todos los clientes; por ejemplo, un único objeto de base de datos compartido por distintas partes del programa.
- El patrón Singleton deshabilita el resto de las maneras de crear objetos de una clase, excepto el método especial de creación. Este método crea un nuevo objeto, o bien devuelve uno existente si ya ha sido creado.
- **Utiliza el patrón Singleton cuando necesites un control más estricto de las variables globales.**
- Al contrario que las variables globales, el patrón Singleton garantiza que haya una única instancia de una clase. A excepción de la propia clase Singleton, nada puede sustituir la instancia en caché.
- Ten en cuenta que siempre podrás ajustar esta limitación y permitir la creación de cierto número de instancias Singleton. La única parte del código que requiere cambios es el cuerpo del método `getInstance`.

Implementación

1. Añade un campo estático privado a la clase para almacenar la instancia Singleton.
2. Declara un método de creación estático público para obtener la instancia Singleton.
3. Implementa una inicialización diferida dentro del método estático. Debe crear un nuevo objeto en su primera llamada y colocarlo dentro del campo estático. El método deberá devolver siempre esa instancia en todas las llamadas siguientes.
4. Declara el constructor de clase como privado. El método estático de la clase seguirá siendo capaz de invocar al constructor, pero no a los otros objetos.
5. Repasa el código cliente y sustituye todas las llamadas directas al constructor de la instancia Singleton por llamadas a su método de creación estático.

Pros y contras

- ✓ Puedes tener la certeza de que una clase tiene una única instancia.
- ✓ Obtienes un punto de acceso global a dicha instancia.
- ✓ El objeto Singleton solo se inicializa cuando se requiere por primera vez.
- ✗ Vulnera el *Principio de responsabilidad única*. El patrón resuelve dos problemas al mismo tiempo.
- ✗ El patrón Singleton puede enmascarar un mal diseño, por ejemplo, cuando los componentes del programa saben demasiado los unos sobre los otros.
- ✗ El patrón requiere de un tratamiento especial en un entorno con múltiples hilos de ejecución, para que varios hilos no creen un objeto Singleton varias veces.
- ✗ Puede resultar complicado realizar la prueba unitaria del código cliente del Singleton porque muchos *frameworks* de prueba dependen de la herencia a la hora de crear objetos simulados (mock objects). Debido a que la clase Singleton es privada y en la mayoría de los lenguajes resulta imposible sobrescribir métodos estáticos, tendrás que pensar en una manera original de simular el Singleton. O, simplemente, no escribas las pruebas. O no utilices el patrón Singleton.

Relaciones con otros patrones

- Una clase **fachada** a menudo puede transformarse en una **Singleton**, ya que un único objeto fachada es suficiente en la mayoría de los casos.
- **Flyweight** podría asemejarse a **Singleton** si de algún modo pudieras reducir todos los estados compartidos de los objetos a un único objeto flyweight. Pero existen dos diferencias fundamentales entre estos patrones:
- Solo debe haber una instancia Singleton, mientras que una clase **Flyweight** puede tener varias instancias con distintos estados intrínsecos.
- El objeto **Singleton** puede ser mutable. Los objetos flyweight son inmutables.
- Los patrones **Abstract Factory**, **Builder** y **Prototype** pueden todos ellos implementarse como **Singletons**.

Ejemplo de una clase Singleton

Supongamos que estamos creando una clase `Logger` que se encargará de registrar mensajes en un archivo de texto. Queremos asegurarnos de que solo haya una instancia de esta clase en todo el programa, así que usamos el patrón Singleton.

```
class Logger:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super(Logger, cls).__new__(cls)
            cls._instance._log_file = "log.txt"
        return cls._instance

    def log(self, message):
        with open(self._log_file, 'a') as f:
            f.write(message + '\n')
```

¿Dónde está la violación del SRP?

En este ejemplo, la clase `Logger` tiene dos responsabilidades:

1. **Responsabilidad de Control de Instancia:** La clase se asegura de que solo haya una instancia de sí misma. Esto se hace en el método `__new__`, donde se controla la creación de la instancia única.
2. **Responsabilidad de Registro de Mensajes:** La clase también se encarga de la lógica de negocio de registrar mensajes en un archivo de texto.

Al mezclar estas dos responsabilidades en una sola clase, la clase `Logger` viola el Principio de Responsabilidad Única. Si en el futuro se decide cambiar cómo se maneja la instancia única (por ejemplo, para permitir múltiples instancias en ciertos casos o cambiar el método de acceso), tendríamos que modificar esta clase, afectando potencialmente la funcionalidad de registro de mensajes.

Solución: Separar Responsabilidades

Una forma de respetar el SRP sería separar estas responsabilidades en diferentes clases:

1. **Clase `Logger`:** Se encarga únicamente de la funcionalidad de registro de mensajes.
2. **Clase `LoggerSingleton`:** Se encarga de manejar la instancia única de `Logger`.

Aquí tienes cómo podría verse esta solución:

```
class Logger:
    def __init__(self):
        self._log_file = "log.txt"

    def log(self, message):
        with open(self._log_file, 'a') as f:
            f.write(message + '\n')

class LoggerSingleton:
    _instance = None

    @classmethod
    def get_instance(cls):
        if cls._instance is None:
            cls._instance = Logger()
        return cls._instance
```

En esta versión:

- La clase Logger se centra únicamente en la lógica de registro.
- La clase LoggerSingleton se encarga de garantizar que solo haya una instancia de Logger, delegando así la responsabilidad de control de instancia.

Ahora, si necesitamos cambiar cómo se maneja la instancia única, solo afectamos a LoggerSingleton, sin tocar la lógica de registro en Logger. Esto respeta el Principio de Responsabilidad Única.

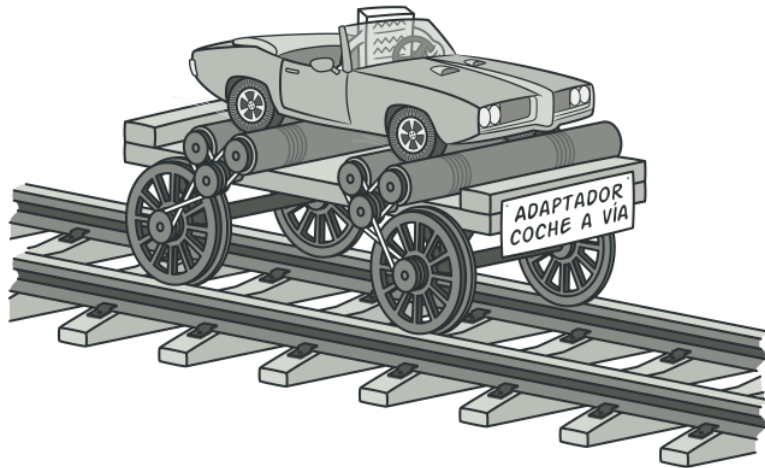
PATRONES ESTRUCTURALES

Adapter

También llamado: Adaptador, Envoltorio, Wrapper

Propósito

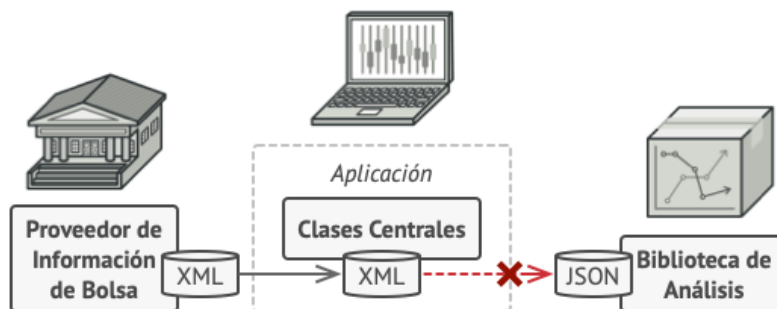
Adapter es un patrón de diseño estructural que permite la colaboración entre objetos con interfaces incompatibles.



Problema

Imagina que estás creando una aplicación de monitoreo del mercado de valores. La aplicación descarga la información de bolsa desde varias fuentes en formato XML para presentarla al usuario con bonitos gráficos y diagramas.

En cierto momento, decides mejorar la aplicación integrando una inteligente biblioteca de análisis de una tercera persona. Pero hay una trampa: la biblioteca de análisis solo funciona con datos en formato JSON.



No puedes utilizar la biblioteca de análisis “tal cual” porque ésta espera los datos en un formato que es incompatible con tu aplicación.

Podrías cambiar la biblioteca para que funcione con XML. Sin embargo, esto podría descomponer parte del código existente que depende de la biblioteca. Y, lo que es peor, podrías no tener siquiera acceso al código fuente de la biblioteca, lo que hace imposible esta solución.

Solución

Puedes crear un *adaptador*. Se trata de un objeto especial que convierte la interfaz de un objeto, de forma que otro objeto pueda comprenderla.

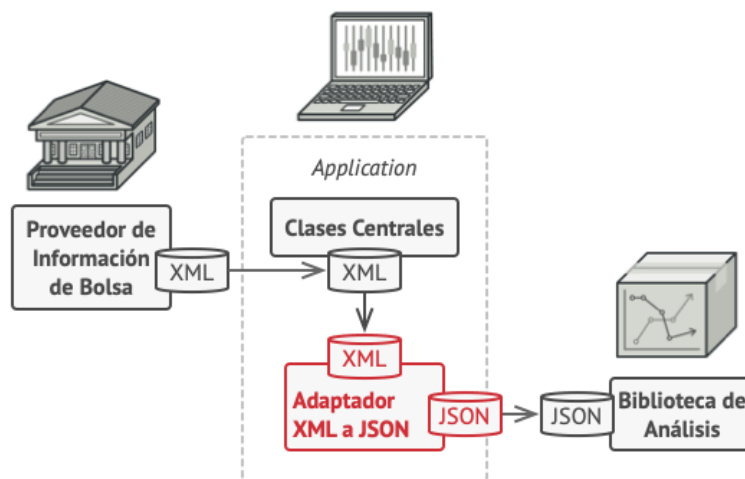
Un adaptador envuelve uno de los objetos para esconder la complejidad de la conversión que tiene lugar tras bambalinas. El objeto envuelto ni siquiera es consciente de la existencia del adaptador. Por ejemplo, puedes

envolver un objeto que opera con metros y kilómetros con un adaptador que convierte todos los datos al sistema anglosajón, es decir, pies y millas.

Los adaptadores no solo convierten datos a varios formatos, sino que también ayudan a objetos con distintas interfaces a colaborar. Funciona así:

1. El adaptador obtiene una interfaz compatible con uno de los objetos existentes.
2. Utilizando esta interfaz, el objeto existente puede invocar con seguridad los métodos del adaptador.
3. Al recibir una llamada, el adaptador pasa la solicitud al segundo objeto, pero en un formato y orden que ese segundo objeto espera.

En ocasiones se puede incluso crear un adaptador de dos direcciones que pueda convertir las llamadas en ambos sentidos.



Regresemos a nuestra aplicación del mercado de valores. Para resolver el dilema de los formatos incompatibles, puedes crear adaptadores de XML a JSON para cada clase de la biblioteca de análisis con la que trabaje tu código directamente. Después ajustas tu código para que se comuniquen con la biblioteca únicamente a través de estos adaptadores. Cuando un adaptador recibe una llamada, traduce los datos XML entrantes a una estructura JSON y pasa la llamada a los métodos adecuados de un objeto de análisis envuelto.

Analogía en el mundo real



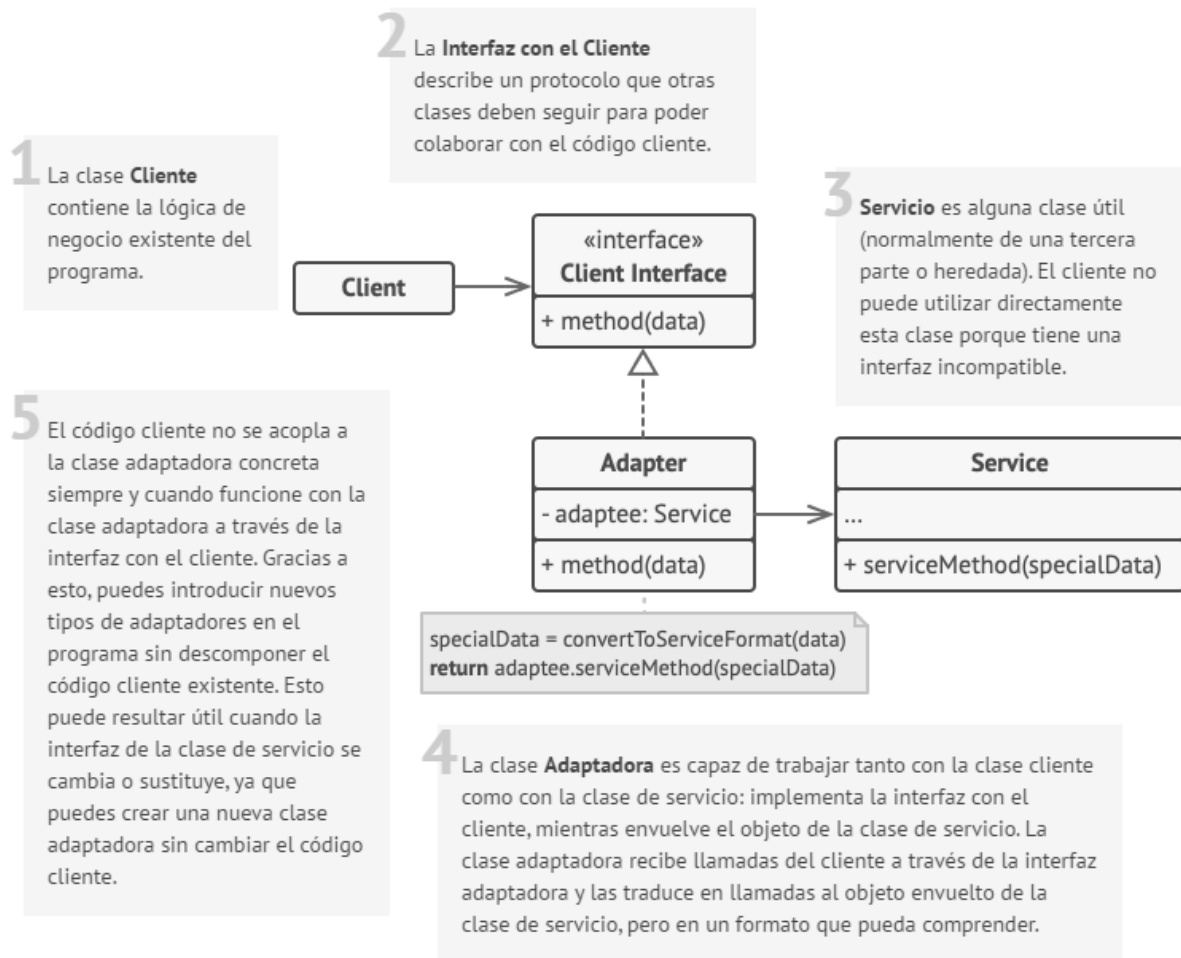
Una maleta antes y después de un viaje al extranjero.

Cuando viajas de Europa a Estados Unidos por primera vez, puede ser que te lleses una sorpresa cuanto intentes cargar tu computadora portátil. Los tipos de enchufe son diferentes en cada país, por lo que un enchufe español no sirve en Estados Unidos. El problema puede solucionarse utilizando un adaptador que incluya el enchufe americano y el europeo.

Estructura

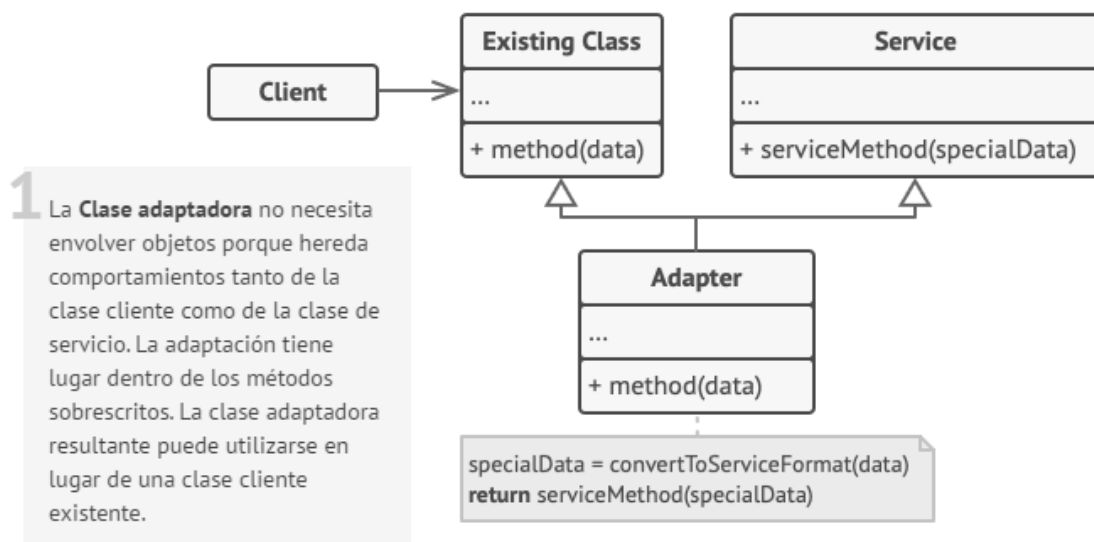
Adaptador de objetos

Esta implementación utiliza el principio de composición de objetos: el adaptador implementa la interfaz de un objeto y envuelve el otro. Puede implementarse en todos los lenguajes de programación populares.



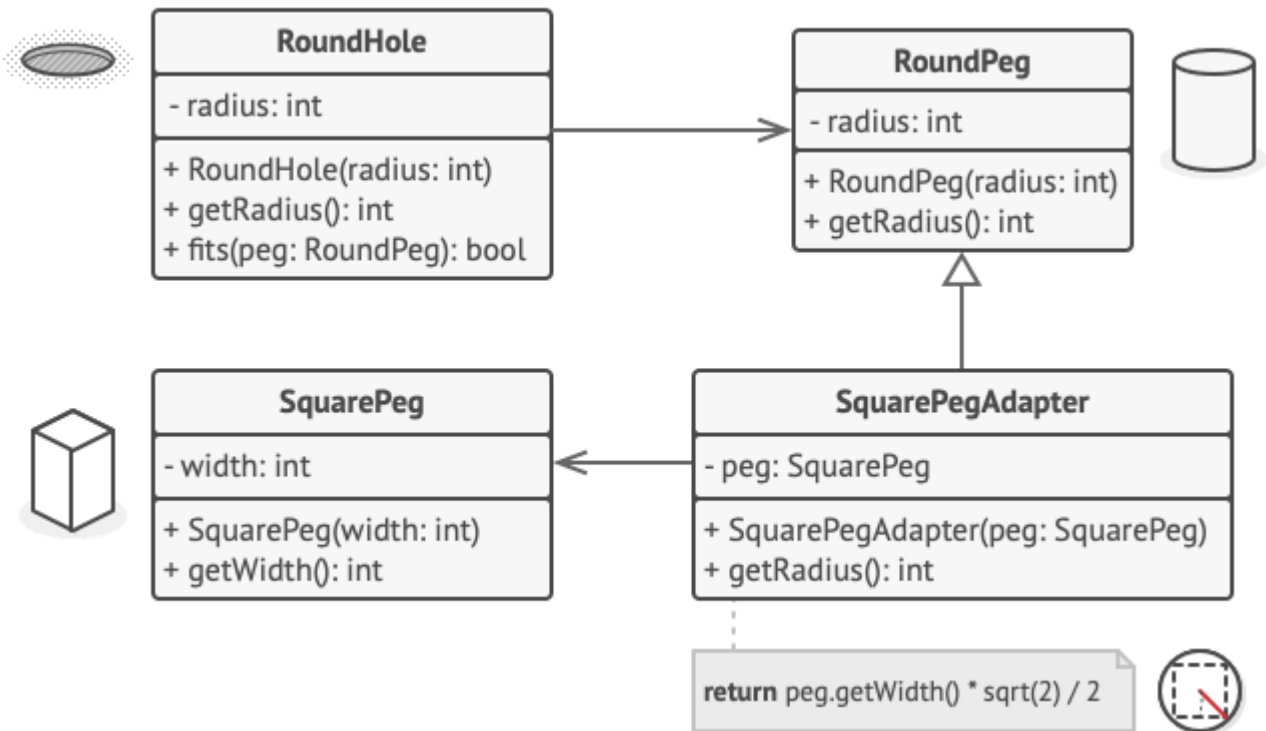
Clase adaptadora

Esta implementación utiliza la herencia, porque la clase adaptadora hereda interfaces de ambos objetos al mismo tiempo. Ten en cuenta que esta opción sólo puede implementarse en lenguajes de programación que soporten la herencia múltiple, como C++.



Pseudocódigo

Este ejemplo del patrón **Adapter** se basa en el clásico conflicto entre piezas cuadradas y agujeros redondos.



Adaptando piezas cuadradas a agujeros redondos.

El patrón Adapter finge ser una pieza redonda con un radio igual a la mitad del diámetro del cuadrado (en otras palabras, el radio del círculo más pequeño en el que quepa la pieza cuadrada).

```

// Digamos que tienes dos clases con interfaces compatibles:
// RoundHole (HoyoRedondo) y RoundPeg (PiezaRedonda).
class RoundHole is
    constructor RoundHole(radius) { ... }

    method getRadius() is
        // Devuelve el radio del agujero.

    method fits(peg: RoundPeg) is
        return this.getRadius() >= peg.getRadius()

class RoundPeg is
    constructor RoundPeg(radius) { ... }

    method getRadius() is
        // Devuelve el radio de la pieza.

// Pero hay una clase incompatible: SquarePeg (PiezaCuadrada).
class SquarePeg is
    constructor SquarePeg(width) { ... }

    method getWidth() is
        // Devuelve la anchura de la pieza cuadrada.

// Una clase adaptadora te permite encajar piezas cuadradas en
// hoyos redondos. Extiende la clase RoundPeg para permitir a
// los objetos adaptadores actuar como piezas redondas.
class SquarePegAdapter extends RoundPeg is
    // En realidad, el adaptador contiene una instancia de la
    // clase SquarePeg.
    private field peg: SquarePeg

    constructor SquarePegAdapter(peg: SquarePeg) is
        this.peg = peg

    method getRadius() is
        // El adaptador simula que es una pieza redonda con un
        // radio que pueda albergar la pieza cuadrada que el
        // adaptador envuelve.
        return peg.getWidth() * Math.sqrt(2) / 2

// En algún punto del código cliente.
hole = new RoundHole(5)
rpeg = new RoundPeg(5)
hole.fits(rpeg) // verdadero

small_sqpeg = new SquarePeg(5)
large_sqpeg = new SquarePeg(10)
hole.fits(small_sqpeg) // esto no compila (tipos incompatibles)

small_sqpeg_adapter = new SquarePegAdapter(small_sqpeg)
large_sqpeg_adapter = new SquarePegAdapter(large_sqpeg)
hole.fits(small_sqpeg_adapter) // verdadero
hole.fits(large_sqpeg_adapter) // falso

```


Aplicabilidad

- **Utiliza la clase adaptadora cuando quieras usar una clase existente, pero cuya interfaz no sea compatible con el resto del código.**
- El patrón Adapter te permite crear una clase intermedia que sirva como traductora entre tu código y una clase heredada, una clase de un tercero o cualquier otra clase con una interfaz extraña.
- **Utiliza el patrón cuando quieras reutilizar varias subclases existentes que carezcan de alguna funcionalidad común que no pueda añadirse a la superclase.**
- Puedes extender cada subclase y colocar la funcionalidad que falta, dentro de las nuevas clases hijas. No obstante, deberás duplicar el código en todas estas nuevas clases, lo cual huele muy mal.

Una solución mucho más elegante sería colocar la funcionalidad que falta dentro de una clase adaptadora. Después puedes envolver objetos a los que les falten funciones, dentro de la clase adaptadora, obteniendo esas funciones necesarias de un modo dinámico. Para que esto funcione, las clases en cuestión deben tener una interfaz común y el campo de la clase adaptadora debe seguir dicha interfaz. Este procedimiento es muy similar al del patrón Decorator.

Implementación

1. Asegúrate de que tienes al menos dos clases con interfaces incompatibles:
 - Una útil clase *servicio* que no puedes cambiar (a menudo de un tercero, heredada o con muchas dependencias existentes).
 - Una o varias clases *cliente* que se beneficiarían de contar con una clase de servicio.
2. Declara la interfaz con el cliente y describe el modo en que las clases cliente se comunican con la clase de servicio.
3. Crea la clase adaptadora y haz que siga la interfaz con el cliente. Deja todos los métodos vacíos por ahora.
4. Añade un campo a la clase adaptadora para almacenar una referencia al objeto de servicio. La práctica común es inicializar este campo a través del constructor, pero en ocasiones es adecuado pasarlo al adaptador cuando se invocan sus métodos.
5. Uno por uno, implementa todos los métodos de la interfaz con el cliente en la clase adaptadora. La clase adaptadora deberá delegar la mayor parte del trabajo real al objeto de servicio, gestionando tan solo la interfaz o la conversión de formato de los datos.
6. Las clases cliente deberán utilizar la clase adaptadora a través de la interfaz con el cliente. Esto te permitirá cambiar o extender las clases adaptadoras sin afectar al código cliente.

Pros y contras

- ✓ *Principio de responsabilidad única.* Puedes separar la interfaz o el código de conversión de datos de la lógica de negocio primaria del programa.
- ✓ *Principio de abierto/cerrado.* Puedes introducir nuevos tipos de adaptadores al programa sin descomponer el código cliente existente, siempre y cuando trabajen con los adaptadores a través de la interfaz con el cliente.
- ✗ La complejidad general del código aumenta, ya que debes introducir un grupo de nuevas interfaces y clases. En ocasiones resulta más sencillo cambiar la clase de servicio de modo que coincida con el resto de tu código.

Relaciones con otros patrones

- **Bridge** suele diseñarse por anticipado, lo que te permite desarrollar partes de una aplicación de forma independiente entre sí. Por otro lado, **Adapter** se utiliza habitualmente con una aplicación existente para hacer que unas clases que de otro modo serían incompatibles, trabajen juntas sin problemas.
- **Adapter** cambia la interfaz de un objeto existente mientras que **Decorator** mejora un objeto sin cambiar su interfaz. Además, *Decorator* soporta la composición recursiva, lo cual no es posible al utilizar *Adapter*.

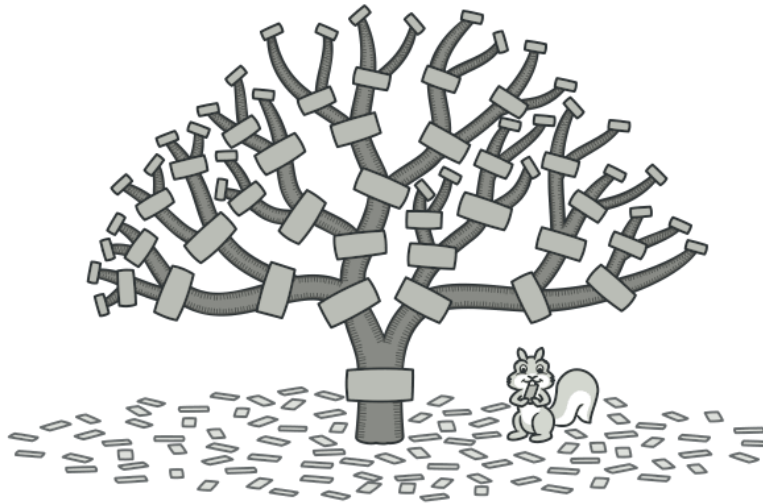
- **Adapter** proporciona una interfaz diferente al objeto envuelto, **Proxy** le proporciona la misma interfaz y **Decorator** le proporciona una interfaz mejorada.
- **Facade** define una nueva interfaz para objetos existentes, mientras que **Adapter** intenta hacer que la interfaz existente sea utilizable. Normalmente *Adapter* sólo envuelve un objeto, mientras que *Facade* trabaja con todo un subsistema de objetos.
- **Bridge**, **State**, **Strategy** (y, hasta cierto punto, **Adapter**) tienen estructuras muy similares. De hecho, todos estos patrones se basan en la composición, que consiste en delegar trabajo a otros objetos. Sin embargo, todos ellos solucionan problemas diferentes. Un patrón no es simplemente una receta para estructurar tu código de una forma específica. También puede comunicar a otros desarrolladores el problema que resuelve.

Composite

También llamado: Objeto compuesto, Object Tree

Propósito

Composite es un patrón de diseño estructural que te permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales.

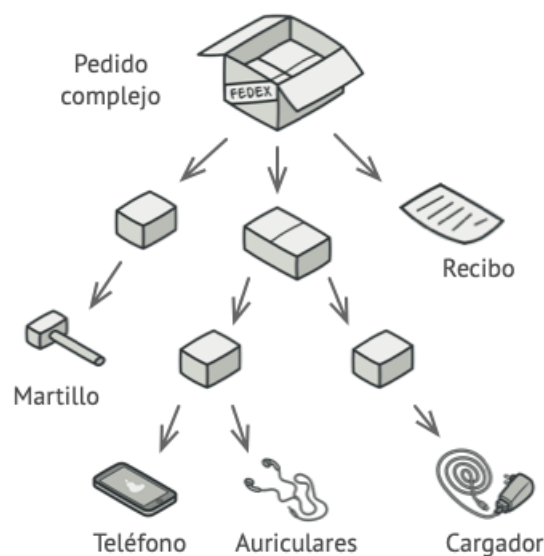


Problema

El uso del patrón Composite sólo tiene sentido cuando el modelo central de tu aplicación puede representarse en forma de árbol.

Por ejemplo, imagina que tienes dos tipos de objetos: **Productos** y **Cajas**. Una **Caja** puede contener varios **Productos** así como cierto número de **Cajas** más pequeñas. Estas **Cajas** pequeñas también pueden contener algunos **Productos** o incluso **Cajas** más pequeñas, y así sucesivamente.

Digamos que decides crear un sistema de pedidos que utiliza estas clases. Los pedidos pueden contener productos sencillos sin envolver, así como cajas llenas de productos... y otras cajas. ¿Cómo determinarás el precio total de ese pedido?



Un pedido puede incluir varios productos empaquetados en cajas, que a su vez están empaquetados en cajas más grandes y así sucesivamente. La estructura se asemeja a un árbol boca abajo.

Puedes intentar la solución directa: desenvolver todas las cajas, repasar todos los productos y calcular el total. Esto sería viable en el mundo real; pero en un programa no es tan fácil como ejecutar un bucle. Tienes que conocer de antemano las clases de `Productos` y `Cajas` a iterar, el nivel de anidación de las cajas y otros detalles desagradables. Todo esto provoca que la solución directa sea demasiado complicada, o incluso imposible.

Solución

El patrón Composite sugiere que trabajes con `Productos` y `Cajas` a través de una interfaz común que declara un método para calcular el precio total.

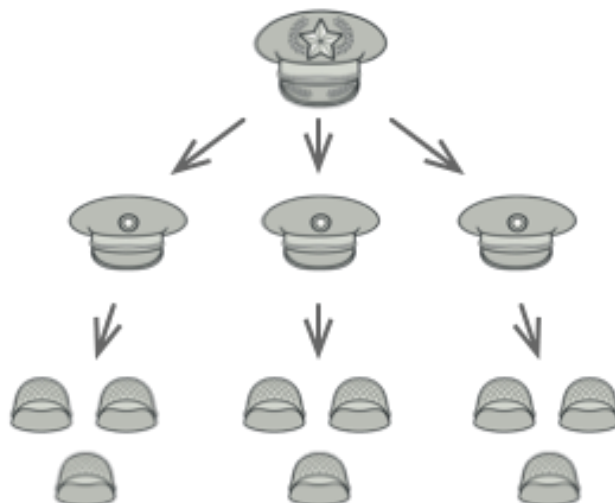
¿Cómo funcionaría este método? Para un producto, sencillamente devuelve el precio del producto. Para una caja, recorre cada artículo que contiene la caja, pregunta su precio y devuelve un total por la caja. Si uno de esos artículos fuera una caja más pequeña, esa caja también comenzaría a repasar su contenido y así sucesivamente, hasta que se calcule el precio de todos los componentes internos. Una caja podría incluso añadir costos adicionales al precio final, como costos de empaquetado.



El patrón Composite te permite ejecutar un comportamiento de forma recursiva sobre todos los componentes de un árbol de objetos.

La gran ventaja de esta solución es que no tienes que preocuparte por las clases concretas de los objetos que componen el árbol. No tienes que saber si un objeto es un producto simple o una sofisticada caja. Puedes tratarlos a todos por igual a través de la interfaz común. Cuando invocas un método, los propios objetos pasan la solicitud a lo largo del árbol.

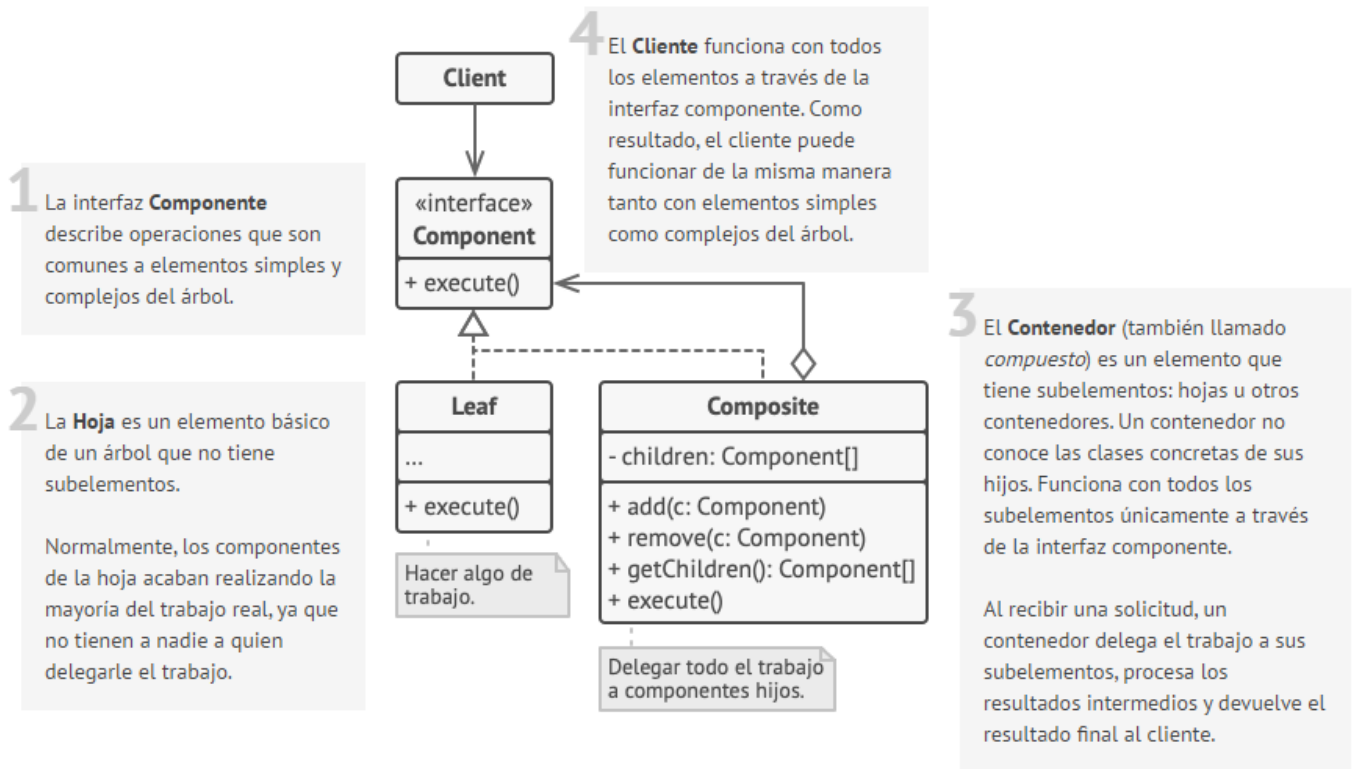
Analogía en el mundo real



Un ejemplo de estructura militar.

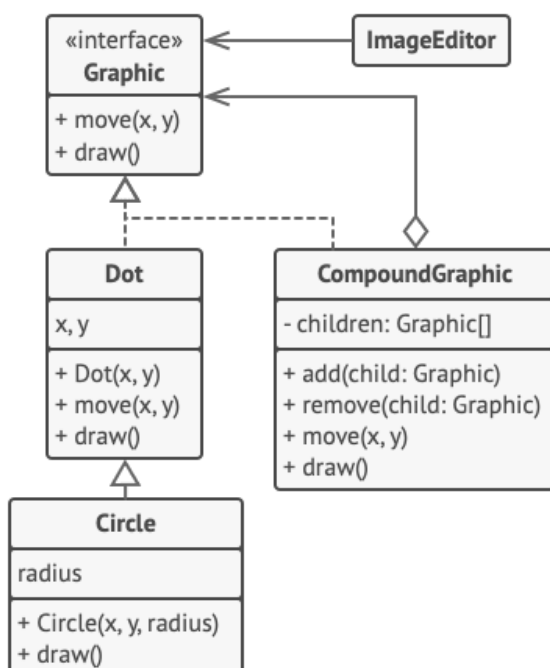
Los ejércitos de la mayoría de países se estructuran como jerarquías. Un ejército está formado por varias divisiones; una división es un grupo de brigadas y una brigada está formada por pelotones, que pueden dividirse en escuadrones. Por último, un escuadrón es un pequeño grupo de soldados reales. Las órdenes se dan en la parte superior de la jerarquía y se pasan hacia abajo por cada nivel hasta que todos los soldados saben lo que hay que hacer.

Estructura



Pseudocódigo

En este ejemplo, el patrón **Composite** te permite implementar el apilamiento (*stacking*) de formas geométricas en un editor gráfico.



Ejemplo del editor de formas geométricas.

La clase **GráficoCompuesto** es un contenedor que puede incluir cualquier cantidad de subformas, incluyendo otras formas compuestas. Una forma compuesta tiene los mismos métodos que una forma simple. Sin embargo, en lugar de hacer algo por su cuenta, una forma compuesta pasa la solicitud de forma recursiva a todos sus hijos y “suma” el resultado.

El código cliente trabaja con todas las formas a través de la interfaz común a todas las clases de forma. De este modo, el cliente no sabe si está trabajando con una forma simple o una compuesta. El cliente puede trabajar con estructuras de objetos muy complejas sin acoplarse a las clases concretas que forman esa estructura.


```

// La interfaz componente declara operaciones comunes para
// objetos simples y complejos de una composición.
interface Graphic is
    method move(x, y)
    method draw()

// La clase hoja representa objetos finales de una composición.
// Un objeto hoja no puede tener ningún subobjeto. Normalmente,
// son los objetos hoja los que hacen el trabajo real, mientras
// que los objetos compuestos se limitan a delegar a sus
// subcomponentes.
class Dot implements Graphic is
    field x, y

    constructor Dot(x, y) { ... }

    method move(x, y) is
        this.x += x, this.y += y

    method draw() is
        // Dibuja un punto en X e Y.

// Todas las clases de componente pueden extender otros
// componentes.
class Circle extends Dot is
    field radius

    constructor Circle(x, y, radius) { ... }

    method draw() is
        // Dibuja un círculo en X y Y con radio R.

// La clase compuesta representa componentes complejos que
// pueden tener hijos. Normalmente los objetos compuestos
// delegan el trabajo real a sus hijos y después "recapitulan"
// el resultado.
class CompoundGraphic implements Graphic is
    field children: array of Graphic

    // Un objeto compuesto puede añadir o eliminar otros
    // componentes (tanto simples como complejos) a o desde su
    // lista hija.
    method add(child: Graphic) is
        // Añade un hijo a la matriz de hijos.

    method remove(child: Graphic) is
        // Elimina un hijo de la matriz de hijos.

    method move(x, y) is
        foreach (child in children) do
            child.move(x, y)

    // Un compuesto ejecuta su lógica primaria de una forma
    // particular. Recorre recursivamente todos sus hijos,
    // recopilando y recapitulando sus resultados. Debido a que
    // los hijos del compuesto pasan esas llamadas a sus propios
    // hijos y así sucesivamente, se recorre todo el árbol de
    // objetos como resultado.
    method draw() is
        // 1. Para cada componente hijo:
        //     - Dibuja el componente.
        //     - Actualiza el rectángulo delimitador.
        // 2. Dibuja un rectángulo de línea punteada utilizando
        // las coordenadas de delimitación.

// El código cliente trabaja con todos los componentes a través
// de su interfaz base. De esta forma el código cliente puede
// soportar componentes de hoja simples así como compuestos
// complejos.
class ImageEditor is
    field all: CompoundGraphic

    method load() is
        all = new CompoundGraphic()
        all.add(new Dot(1, 2))
        all.add(new Circle(5, 3, 10))
        // ...

    // Combina componentes seleccionados para formar un
    // componente compuesto complejo.
    method groupSelected(components: array of Graphic) is
        group = new CompoundGraphic()
        foreach (component in components) do
            group.add(component)
            all.remove(component)
        all.add(group)
        // Se dibujarán todos los componentes.
        all.draw()

```

Aplicabilidad

- ✓ **Utiliza el patrón Composite cuando tengas que implementar una estructura de objetos con forma de árbol.**

El patrón Composite te proporciona dos tipos de elementos básicos que comparten una interfaz común: hojas simples y contenedores complejos. Un contenedor puede estar compuesto por hojas y por otros contenedores. Esto te permite construir una estructura de objetos recursivos anidados parecida a un árbol.

- ✓ **Utiliza el patrón cuando quieras que el código cliente trate elementos simples y complejos de la misma forma.**

Todos los elementos definidos por el patrón Composite comparten una interfaz común. Utilizando esta interfaz, el cliente no tiene que preocuparse por la clase concreta de los objetos con los que funciona.

Implementación

1. Asegúrate de que el modelo central de tu aplicación pueda representarse como una estructura de árbol. Intenta dividirlo en elementos simples y contenedores. Recuerda que los contenedores deben ser capaces de contener tanto elementos simples como otros contenedores.
2. Declara la interfaz componente con una lista de métodos que tengan sentido para componentes simples y complejos.
3. Crea una clase hoja para representar elementos simples. Un programa puede tener varias clases hoja diferentes.
4. Crea una clase contenedora para representar elementos complejos. Incluye un campo matriz en esta clase para almacenar referencias a subelementos. La matriz debe poder almacenar hojas y contenedores, así que asegúrate de declararla con el tipo de la interfaz componente.

Al implementar los métodos de la interfaz componente, recuerda que un contenedor debe delegar la mayor parte del trabajo a los subelementos.

5. Por último, define los métodos para añadir y eliminar elementos hijos dentro del contenedor.

Ten en cuenta que estas operaciones se pueden declarar en la interfaz componente. Esto violaría el *Principio de segregación de la interfaz* porque los métodos de la clase hoja estarían vacíos. No obstante, el cliente podrá tratar a todos los elementos de la misma manera, incluso al componer el árbol.

Pros y contras

- ✓ Puedes trabajar con estructuras de árbol complejas con mayor comodidad: utiliza el polimorfismo y la recursión en tu favor.
- ✓ *Principio de abierto/cerrado*. Puedes introducir nuevos tipos de elemento en la aplicación sin descomponer el código existente, que ahora funciona con el árbol de objetos.
- ✗ Puede resultar difícil proporcionar una interfaz común para clases cuya funcionalidad difiere demasiado. En algunos casos, tendrás que generalizar en exceso la interfaz componente, provocando que sea más difícil de comprender.

Relaciones con otros patrones

- Puedes utilizar **Builder** al crear árboles **Composite** complejos porque puedes programar sus pasos de construcción para que funcionen de forma recursiva.
- **Chain of Responsibility** se utiliza a menudo junto a **Composite**. En este caso, cuando un componente hoja recibe una solicitud, puede pasarla a lo largo de la cadena de todos los componentes padre hasta la raíz del árbol de objetos.
- Puedes utilizar **Iteradores** para recorrer árboles **Composite**.
- Puedes utilizar el patrón **Visitor** para ejecutar una operación sobre un árbol **Composite** entero.
- Puedes implementar nodos de hoja compartidos del árbol **Composite** como **Flyweights** para ahorrar memoria RAM.
- **Composite** y **Decorator** tienen diagramas de estructura similares ya que ambos se basan en la composición recursiva para organizar un número indefinido de objetos.

Un *Decorator* es como un *Composite* pero sólo tiene un componente hijo. Hay otra diferencia importante: *Decorator* añade responsabilidades adicionales al objeto envuelto, mientras que *Composite* se limita a “recapitular” los resultados de sus hijos.

No obstante, los patrones también pueden colaborar: puedes utilizar el *Decorator* para extender el comportamiento de un objeto específico del árbol *Composite*.

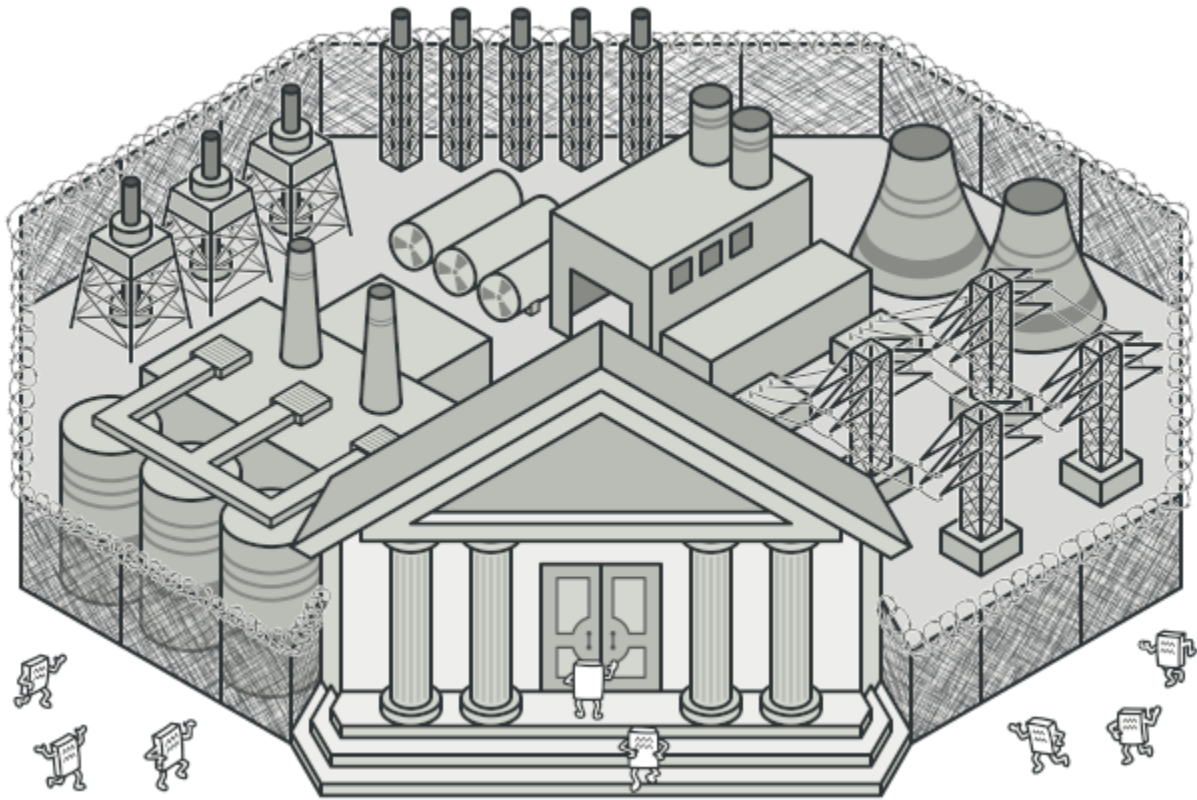
- Los diseños que hacen un uso amplio de **Composite** y **Decorator** a menudo pueden beneficiarse del uso del **Prototype**. Aplicar el patrón te permite clonar estructuras complejas en lugar de reconstruirlas desde cero.

Facade

También llamado: Fachada

Propósito

Facade es un patrón de diseño estructural que proporciona una interfaz simplificada a una biblioteca, un framework o cualquier otro grupo complejo de clases.



Problema

Imagina que debes lograr que tu código trabaje con un amplio grupo de objetos que pertenecen a una sofisticada biblioteca o *framework*. Normalmente, debes inicializar todos esos objetos, llevar un registro de las dependencias, ejecutar los métodos en el orden correcto y así sucesivamente.

Como resultado, la lógica de negocio de tus clases se vería estrechamente acoplada a los detalles de implementación de las clases de terceros, haciéndola difícil de comprender y mantener.

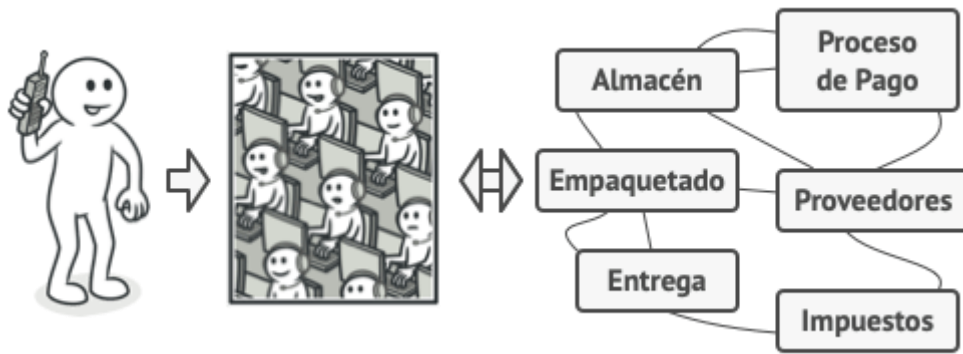
Solución

Una fachada es una clase que proporciona una interfaz simple a un subsistema complejo que contiene muchas partes móviles. Una fachada puede proporcionar una funcionalidad limitada en comparación con trabajar directamente con el subsistema. Sin embargo, tan solo incluye las funciones realmente importantes para los clientes.

Tener una fachada resulta útil cuando tienes que integrar tu aplicación con una biblioteca sofisticada con decenas de funciones, de la cual sólo necesitas una pequeña parte.

Por ejemplo, una aplicación que sube breves vídeos divertidos de gatos a las redes sociales, podría potencialmente utilizar una biblioteca de conversión de vídeo profesional. Sin embargo, lo único que necesita en realidad es una clase con el método simple `codificar(nombreDelArchivo, formato)`. Una vez que crees dicha clase y la conectes con la biblioteca de conversión de vídeo, tendrás tu primera fachada.

Analogía en el mundo real



Haciendo pedidos por teléfono.

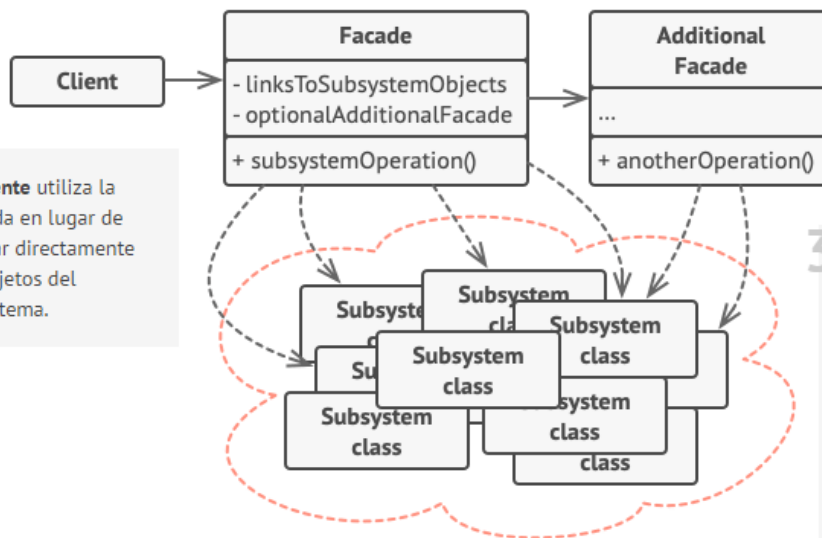
Cuando llamas a una tienda para hacer un pedido por teléfono, un operador es tu fachada a todos los servicios y departamentos de la tienda. El operador te proporciona una sencilla interfaz de voz al sistema de pedidos, pasarelas de pago y varios servicios de entrega.

Estructura

1 El patrón **Facade** proporciona un práctico acceso a una parte específica de la funcionalidad del subsistema. Sabe a dónde dirigir la petición del cliente y cómo operar todas las partes móviles.

2 Puede crearse una clase **Fachada Adicional** para evitar contaminar una única fachada con funciones no relacionadas que podrían convertirla en otra estructura compleja. Las fachadas adicionales pueden utilizarse por clientes y por otras fachadas.

4 El **Cliente** utiliza la fachada en lugar de invocar directamente los objetos del subsistema.

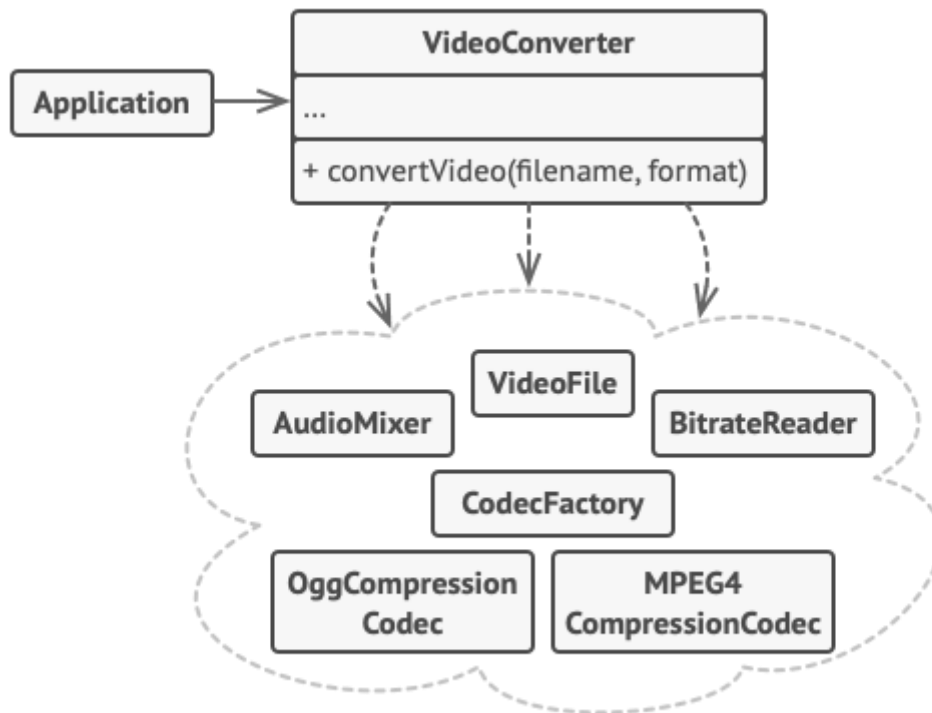


3 El **Subsistema Complejo** consiste en decenas de objetos diversos. Para lograr que todos hagan algo significativo, debes profundizar en los detalles de implementación del subsistema, que pueden incluir inicializar objetos en el orden correcto y suministrarles datos en el formato adecuado.

Las clases del subsistema no conocen la existencia de la fachada. Operan dentro del sistema y trabajan entre sí directamente.

Pseudocódigo

En este ejemplo, el patrón **Facade** simplifica la interacción con un framework complejo de conversión de vídeo.



Un ejemplo de aislamiento de múltiples dependencias dentro de una única clase fachada.

En lugar de hacer que tu código trabaje con decenas de las clases del framework directamente, creas una clase fachada que encapsula esa funcionalidad y la esconde del resto del código. Esta estructura también te ayuda a minimizar el esfuerzo de actualizar a futuras versiones del framework o de sustituirlo por otro. Lo único que tendrías que cambiar en la aplicación es la implementación de los métodos de la fachada.

```

// Estas son algunas de las clases de un framework de conversión
// de video de un tercero. No controlamos ese código, por lo que
// no podemos simplificarlo.

class VideoFile
// ...

class OggCompressionCodec
// ...

class MPEG4CompressionCodec
// ...

class CodecFactory
// ...

class BitrateReader
// ...

class AudioMixer
// ...

// Creamos una clase fachada para esconder la complejidad del
// framework tras una interfaz simple. Es una solución de
// equilibrio entre funcionalidad y simplicidad.
class VideoConverter is
    method convert(filename, format):File is
        file = new VideoFile(filename)
        sourceCodec = (new CodecFactory).extract(file)
        if (format == "mp4")
            destinationCodec = new MPEG4CompressionCodec()
        else
            destinationCodec = new OggCompressionCodec()
        buffer = BitrateReader.read(filename, sourceCodec)
        result = BitrateReader.convert(buffer, destinationCodec)
        result = (new AudioMixer()).fix(result)
        return new File(result)

// Las clases Application no dependen de un millón de clases
// proporcionadas por el complejo framework. Además, si decides
// cambiar los frameworks, sólo tendrás de volver a escribir la
// clase fachada.
class Application is
    method main() is
        convertor = new VideoConverter()
        mp4 = convertor.convert("funny-cats-video.ogg", "mp4")
        mp4.save()

```

Aplicabilidad

- ✓ **Utiliza el patrón Facade cuando necesites una interfaz limitada pero directa a un subsistema complejo.**

A menudo los subsistemas se vuelven más complejos con el tiempo. Incluso la aplicación de patrones de diseño suele conducir a la creación de un mayor número de clases. Un subsistema puede hacerse más flexible y más fácil de reutilizar en varios contextos, pero la cantidad de código de configuración que exige de un cliente, crece aún más. El patrón Facade intenta solucionar este problema proporcionando un atajo a las funciones más utilizadas del subsistema que mejor encajan con los requisitos del cliente.

- ✓ **Utiliza el patrón Facade cuando quieras estructurar un subsistema en capas.**

Crea fachadas para definir puntos de entrada a cada nivel de un subsistema. Puedes reducir el acoplamiento entre varios subsistemas exigiéndoles que se comuniquen únicamente mediante fachadas.

Por ejemplo, regresemos a nuestro framework de conversión de vídeo. Puede dividirse en dos capas: la relacionada con el vídeo y la relacionada con el audio. Puedes crear una fachada para cada capa y hacer que las clases de cada una de ellas se comuniquen entre sí a través de esas fachadas. Este procedimiento es bastante similar al patrón Mediator.

Implementación

1. Comprueba si es posible proporcionar una interfaz más simple que la que está proporcionando un subsistema existente. Estás bien encaminado si esta interfaz hace que el código cliente sea independiente de muchas de las clases del subsistema.
2. Declara e implementa esta interfaz en una nueva clase fachada. La fachada deberá redireccionar las llamadas desde el código cliente a los objetos adecuados del subsistema. La fachada deberá ser responsable de inicializar el subsistema y gestionar su ciclo de vida, a no ser que el código cliente ya lo haga.
3. Para aprovechar el patrón al máximo, haz que todo el código cliente se comunique con el subsistema únicamente a través de la fachada. Ahora el código cliente está protegido de cualquier cambio en el código del subsistema. Por ejemplo, cuando se actualice un subsistema a una nueva versión, sólo tendrás que modificar el código de la fachada.
4. Si la fachada se vuelve demasiado grande, piensa en extraer parte de su comportamiento y colocarlo dentro de una nueva clase fachada refinada.

Pros y contras

- ✓ Puedes aislar tu código de la complejidad de un subsistema.
- ✗ Una fachada puede convertirse en un objeto todopoderoso acoplado a todas las clases de una aplicación.

Relaciones con otros patrones

- Facade define una nueva interfaz para objetos existentes, mientras que Adapter intenta hacer que la interfaz existente sea utilizable. Normalmente *Adapter* sólo envuelve un objeto, mientras que *Facade* trabaja con todo un subsistema de objetos.
- Abstract Factory puede servir como alternativa a Facade cuando tan solo deseas esconder la forma en que se crean los objetos del subsistema a partir del código cliente.
- Flyweight muestra cómo crear muchos pequeños objetos, mientras que Facade muestra cómo crear un único objeto que represente un subsistema completo.
- Facade y Mediator tienen trabajos similares: ambos intentan organizar la colaboración entre muchas clases estrechamente acopladas.

- *Facade* define una interfaz simplificada a un subsistema de objetos, pero no introduce ninguna nueva funcionalidad. El propio subsistema no conoce la fachada. Los objetos del subsistema pueden comunicarse directamente.
 - *Mediator* centraliza la comunicación entre componentes del sistema. Los componentes conocen únicamente el objeto mediador y no se comunican directamente.
- Una clase **fachada** a menudo puede transformarse en una **Singleton**, ya que un único objeto fachada es suficiente en la mayoría de los casos.
- **Facade** es similar a **Proxy** en el sentido de que ambos pueden almacenar temporalmente una entidad compleja e inicializarla por su cuenta. Al contrario que *Facade*, *Proxy* tiene la misma interfaz que su objeto de servicio, lo que hace que sean intercambiables.

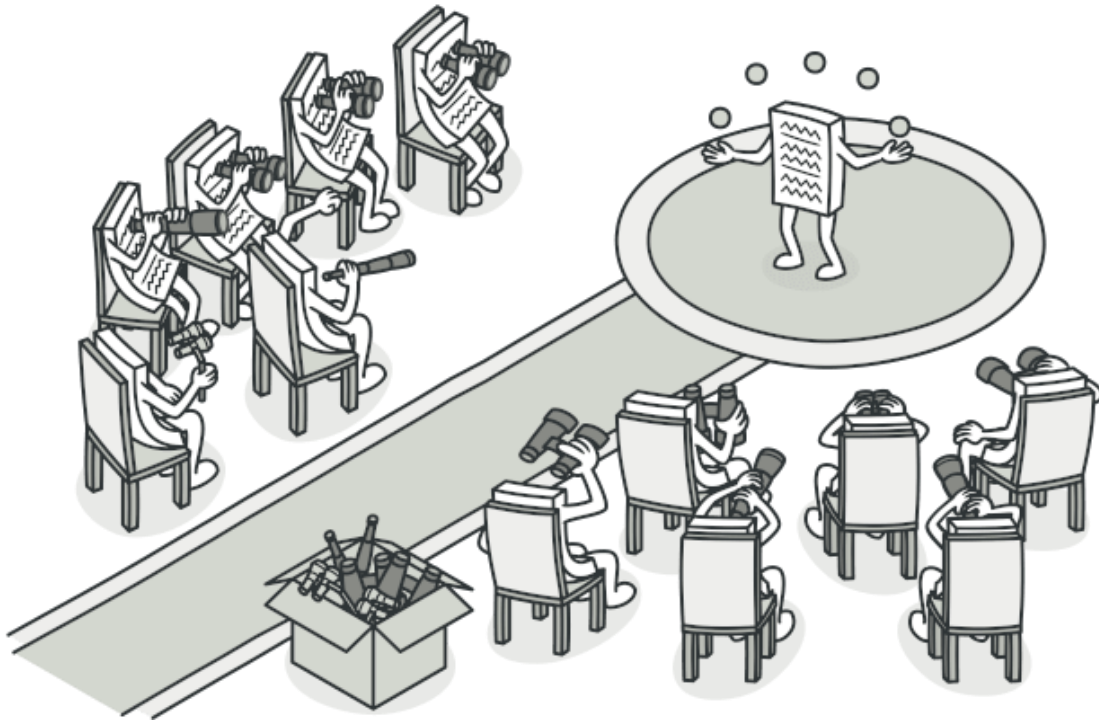
PATRONES DE COMPORTAMIENTO

Observer

También llamado: Observador, Publicación-Suscripción, Modelo-patrón, Event-Subscriber, Listener

Propósito

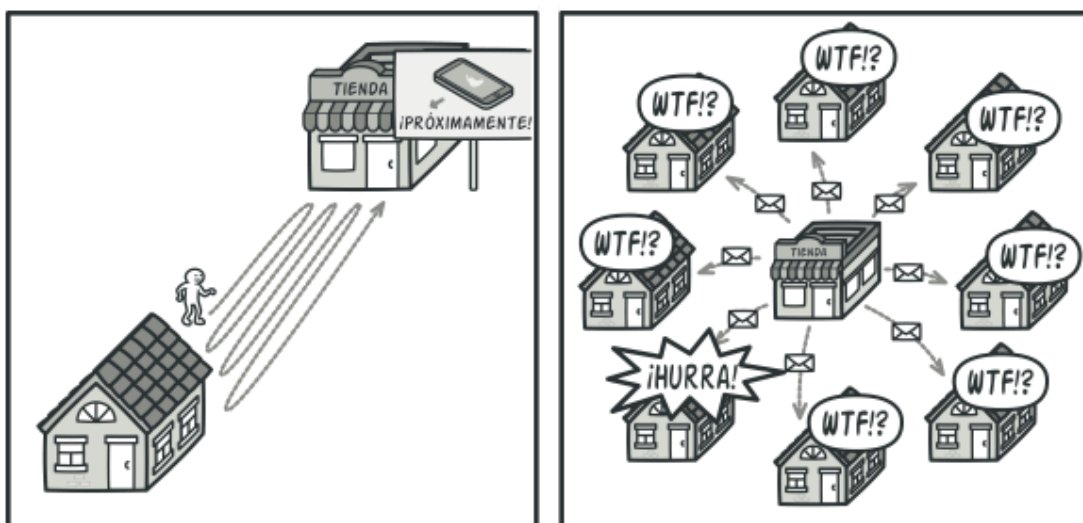
Observer es un patrón de diseño de comportamiento que te permite definir un mecanismo de suscripción para notificar a varios objetos sobre cualquier evento que le suceda al objeto que están observando.



Problema

Imagina que tienes dos tipos de objetos: un objeto **Cliente** y un objeto **Tienda**. El cliente está muy interesado en una marca particular de producto (digamos, un nuevo modelo de iPhone) que estará disponible en la tienda muy pronto.

El cliente puede visitar la tienda cada día para comprobar la disponibilidad del producto. Pero, mientras el producto está en camino, la mayoría de estos viajes serán en vano.



Visita a la tienda vs. envío de spam

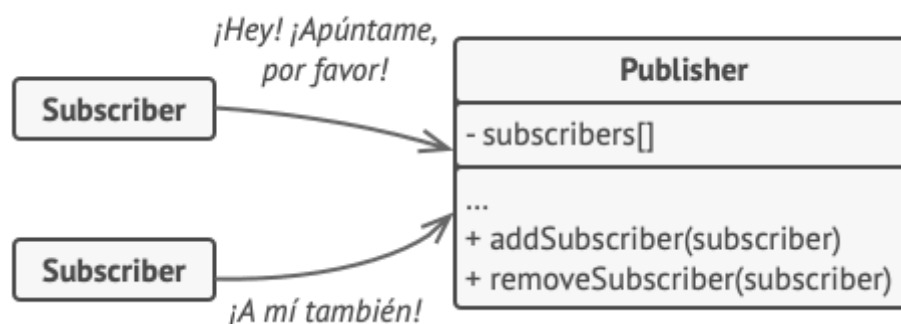
Por otro lado, la tienda podría enviar cientos de correos (lo cual se podría considerar spam) a todos los clientes cada vez que hay un nuevo producto disponible. Esto ahorraría a los clientes los interminables viajes a la tienda, pero, al mismo tiempo, molestaría a otros clientes que no están interesados en los nuevos productos.

Parece que nos encontramos ante un conflicto. O el cliente pierde tiempo comprobando la disponibilidad del producto, o bien la tienda desperdicia recursos notificando a los clientes equivocados.

Solución

El objeto que tiene un estado interesante suele denominarse *sujeto*, pero, como también va a notificar a otros objetos los cambios en su estado, le llamaremos *notificador* (en ocasiones también llamado *publicador*). El resto de los objetos que quieren conocer los cambios en el estado del notificador, se denominan *suscriptores*.

El patrón Observer sugiere que añadas un mecanismo de suscripción a la clase notificadora para que los objetos individuales puedan suscribirse o cancelar su suscripción a un flujo de eventos que proviene de esa notificadora. ¡No temas! No es tan complicado como parece. En realidad, este mecanismo consiste en: 1) un campo matriz para almacenar una lista de referencias a objetos suscriptores y 2) varios métodos públicos que permiten añadir suscriptores y eliminarlos de esa lista.

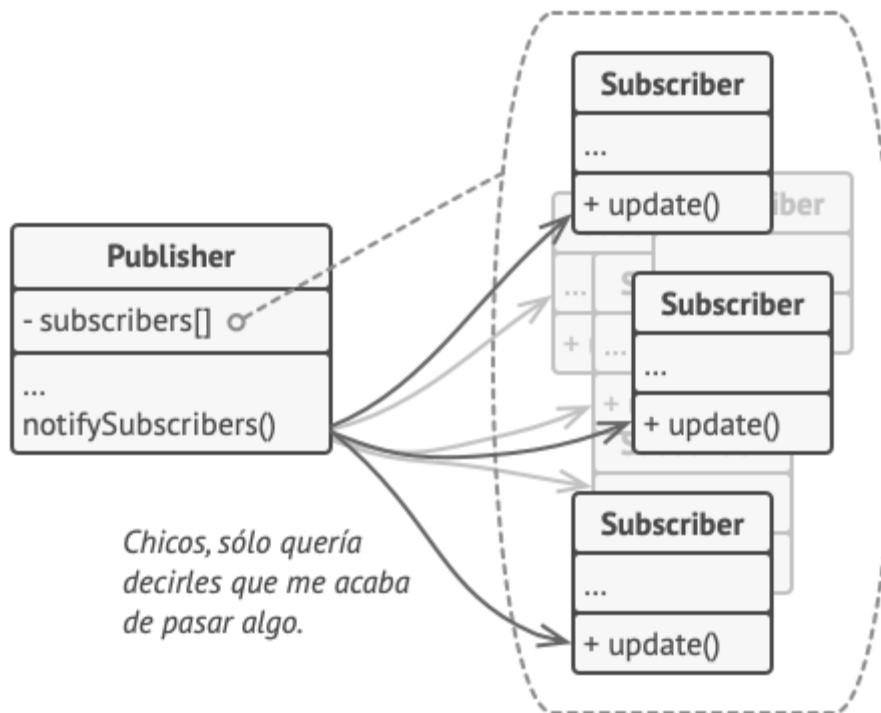


Un mecanismo de suscripción permite a los objetos individuales suscribirse a notificaciones de eventos.

Ahora, cuando le sucede un evento importante al notificador, recorre sus suscriptores y llama al método de notificación específico de sus objetos.

Las aplicaciones reales pueden tener decenas de clases suscriptoras diferentes interesadas en seguir los eventos de la misma clase notificadora. No querrás acoplar la notificadora a todas esas clases. Además, puede que no conozcas algunas de ellas de antemano si se supone que otras personas pueden utilizar tu clase notificadora.

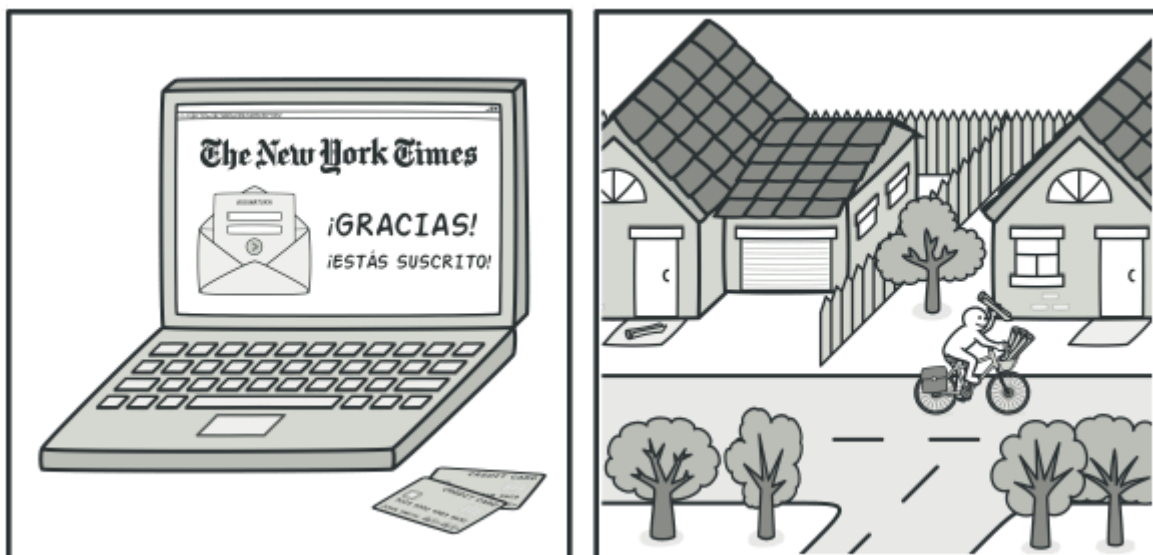
Por eso es fundamental que todos los suscriptores implementen la misma interfaz y que el notificador únicamente se comunique con ellos a través de esa interfaz. Esta interfaz debe declarar el método de notificación junto con un grupo de parámetros que el notificador puede utilizar para pasar cierta información contextual con la notificación.



El notificador notifica a los suscriptores invocando el método de notificación específico de sus objetos.

Si tu aplicación tiene varios tipos diferentes de notificadores y quieres hacer a tus suscriptores compatibles con todos ellos, puedes ir más allá y hacer que todos los notificadores sigan la misma interfaz. Esta interfaz sólo tendrá que describir algunos métodos de suscripción. La interfaz permitirá a los suscriptores observar los estados de los notificadores sin acoplarse a sus clases concretas.

Analogía en el mundo real

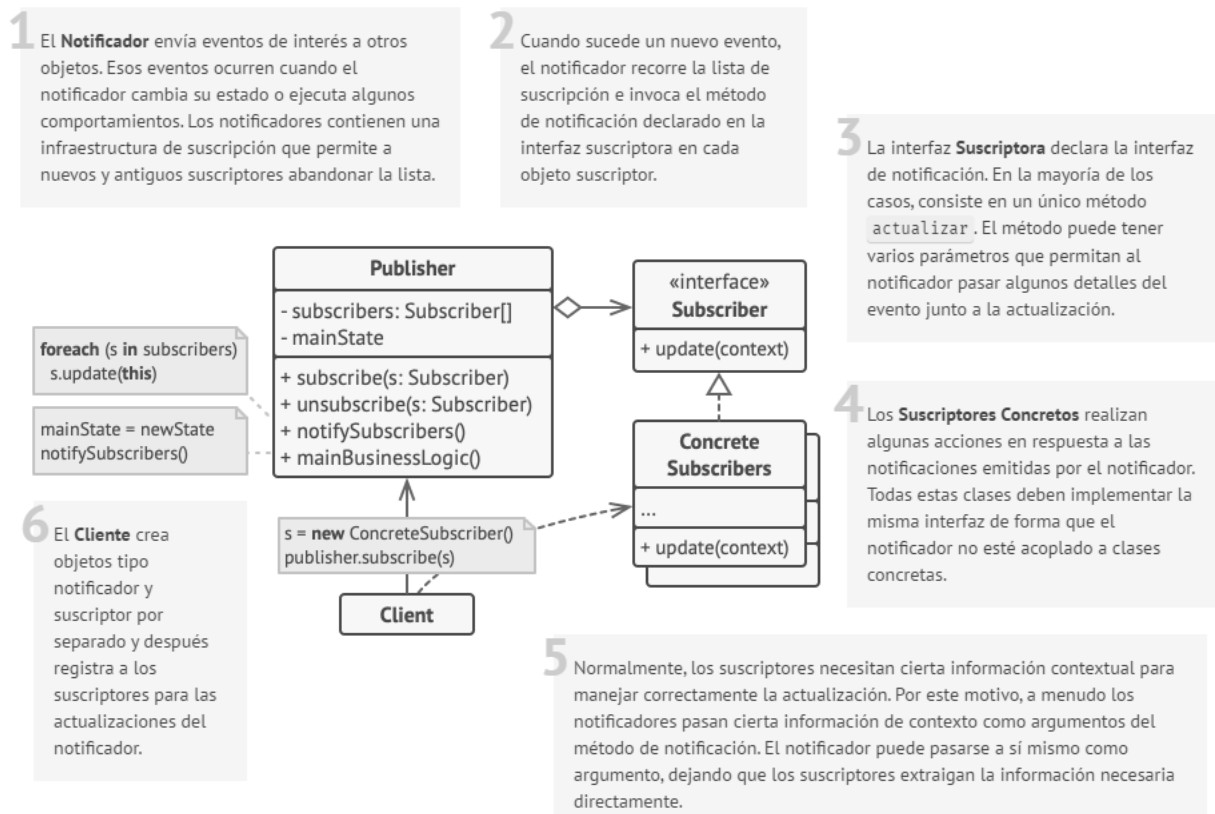


Suscripciones a revistas y periódicos.

Si te suscribes a un periódico o una revista, ya no necesitarás ir a la tienda a comprobar si el siguiente número está disponible. En lugar de eso, el notificador envía nuevos números directamente a tu buzón justo después de la publicación, o incluso antes.

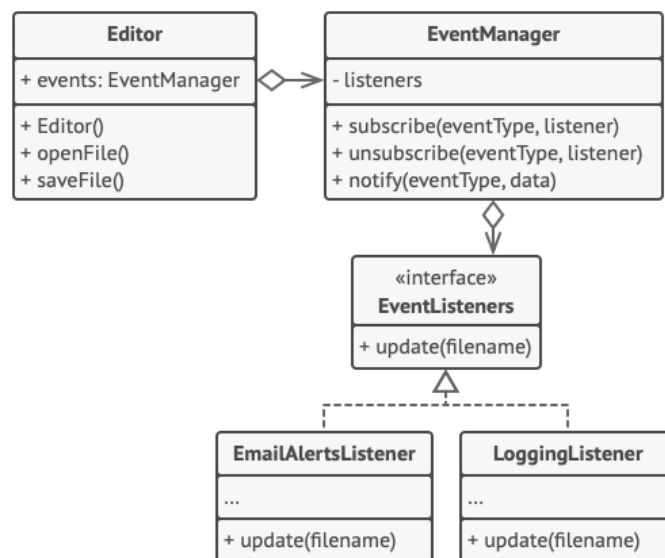
El notificador mantiene una lista de suscriptores y sabe qué revistas les interesan. Los suscriptores pueden abandonar la lista en cualquier momento si quieren que el notificador deje de enviarles nuevos números.

Estructura



Pseudocódigo

En este ejemplo, el patrón **Observer** permite al objeto editor de texto notificar a otros objetos tipo servicio sobre los cambios en su estado.



Notificar a objetos sobre eventos que suceden a otros objetos.

La lista de suscriptores se compila dinámicamente: los objetos pueden empezar o parar de escuchar notificaciones durante el tiempo de ejecución, dependiendo del comportamiento que desees para tu aplicación.

En esta implementación, la clase editora no mantiene la lista de suscripción por sí misma. Delega este trabajo al objeto ayudante especial dedicado justo a eso. Puedes actualizar ese objeto para que sirva como despachador centralizado de eventos, dejando que cualquier objeto actúe como notificador.

Añadir nuevos suscriptores al programa no requiere cambios en clases notificadoras existentes, siempre y cuando trabajen con todos los suscriptores a través de la misma interfaz.

```

// La clase notificador base incluye código de gestión de
// suscripciones y métodos de notificación.
class EventManager is
    private field listeners: hash map of event types and listeners

    method subscribe(eventType, listener) is
        listeners.add(eventType, listener)

    method unsubscribe(eventType, listener) is
        listeners.remove(eventType, listener)

    method notify(eventType, data) is
        foreach (listener in listeners.of(eventType)) do
            listener.update(data)

// El notificador concreto contiene lógica de negocio real, de
// interés para algunos suscriptores. Podemos derivar esta clase
// de la notificador base, pero esto no siempre es posible en
// el mundo real porque puede que la notificador concreta sea
// ya una subclase. En este caso, puedes modificar la lógica de
// la suscripción con composición, como hicimos aquí.
class Editor is
    public field events: EventManager
    private field file: File

    constructor Editor() is
        events = new EventManager()

    // Los métodos de la lógica de negocio pueden notificar los
    // cambios a los suscriptores.
    method openFile(path) is
        this.file = new File(path)
        events.notify("open", file.name)

    method saveFile() is
        file.write()
        events.notify("save", file.name)

    // ...

// Aquí está la interfaz suscriptora. Si tu lenguaje de
// programación soporta tipos funcionales, puedes sustituir toda
// la jerarquía suscriptora por un grupo de funciones.

interface EventListener is
    method update(filename)

// Los suscriptores concretos reaccionan a las actualizaciones
// emitidas por el notificador al que están unidos.
class LoggingListener implements EventListener is
    private field log: File
    private field message: string

    constructor LoggingListener(log_filename, message) is
        this.log = new File(log_filename)
        this.message = message

    method update(filename) is
        log.write(replace('%s', filename, message))

class EmailAlertsListener implements EventListener is
    private field email: string
    private field message: string

    constructor EmailAlertsListener(email, message) is
        this.email = email
        this.message = message

    method update(filename) is
        system.email(email, replace('%s', filename, message))

// Una aplicación puede configurar notificadores y suscriptores
// durante el tiempo de ejecución.
class Application is
    method config() is
        editor = new Editor()

        logger = new LoggingListener(
            "/path/to/log.txt",
            "Someone has opened the file: %s")
        editor.events.subscribe("open", logger)

        emailAlerts = new EmailAlertsListener(
            "admin@example.com",
            "Someone has changed the file: %s")
        editor.events.subscribe("save", emailAlerts)

```

Aplicabilidad

Utiliza el patrón Observer cuando los cambios en el estado de un objeto puedan necesitar cambiar otros objetos y el grupo de objetos sea desconocido de antemano o cambie dinámicamente.

Puedes experimentar este problema a menudo al trabajar con clases de la interfaz gráfica de usuario. Por ejemplo, si creaste clases personalizadas de botón y quieres permitir al cliente colgar código cliente de tus botones para que se active cuando un usuario pulse un botón.

El patrón Observer permite que cualquier objeto que implemente la interfaz suscriptora pueda suscribirse a notificaciones de eventos en objetos notificadores. Puedes añadir el mecanismo de suscripción a tus botones, permitiendo a los clientes acoplar su código personalizado a través de clases suscriptoras personalizadas.

Utiliza el patrón cuando algunos objetos de tu aplicación deban observar a otros, pero sólo durante un tiempo limitado o en casos específicos.

La lista de suscripción es dinámica, por lo que los suscriptores pueden unirse o abandonar la lista cuando lo deseen.

Implementación

1. Repasa tu lógica de negocio e intenta dividirla en dos partes: la funcionalidad central, independiente del resto de código, actuará como notificador; el resto se convertirá en un grupo de clases suscriptoras.
2. Declara la interfaz suscriptora. Como mínimo, deberá declarar un único método `actualizar`.
3. Declara la interfaz notificadora y describe un par de métodos para añadir y eliminar de la lista un objeto suscriptor. Recuerda que los notificadores deben trabajar con suscriptores únicamente a través de la interfaz suscriptora.
4. Decide dónde colocar la lista de suscripción y la implementación de métodos de suscripción. Normalmente, este código tiene el mismo aspecto para todos los tipos de notificadores, por lo que el lugar obvio para colocarlo es en una clase abstracta derivada directamente de la interfaz notificadora. Los notificadores concretos extienden esa clase, heredando el comportamiento de suscripción.
Sin embargo, si estás aplicando el patrón a una jerarquía de clases existentes, considera una solución basada en la composición: coloca la lógica de la suscripción en un objeto separado y haz que todos los notificadores reales la utilicen.
5. Crea clases notificadoras concretas. Cada vez que suceda algo importante dentro de una notificadora, deberá notificar a todos sus suscriptores.
6. Implementa los métodos de notificación de actualizaciones en clases suscriptoras concretas. La mayoría de las suscriptoras necesitarán cierta información de contexto sobre el evento, que puede pasarse como argumento del método de notificación.
Pero hay otra opción. Al recibir una notificación, el suscriptor puede extraer la información directamente de ella. En este caso, el notificador debe pasarse a sí mismo a través del método de actualización. La opción menos flexible es vincular un notificador con el suscriptor de forma permanente a través del constructor.
7. El cliente debe crear todos los suscriptores necesarios y registrarlos con los notificadores adecuados.

Pros y contras

- ✓ *Principio de abierto/cerrado.* Puedes introducir nuevas clases suscriptoras sin tener que cambiar el código de la notificadora (y viceversa si hay una interfaz notificadora).
- ✓ Puedes establecer relaciones entre objetos durante el tiempo de ejecución.
- ✗ Los suscriptores son notificados en un orden aleatorio.

Relaciones con otros patrones

- **Chain of Responsibility**, **Command**, **Mediator** y **Observer** abordan distintas formas de conectar emisores y receptores de solicitudes:
 - *Chain of Responsibility* pasa una solicitud secuencialmente a lo largo de una cadena dinámica de receptores potenciales hasta que uno de ellos la gestiona.

- *Command* establece conexiones unidireccionales entre emisores y receptores.
- *Mediator* elimina las conexiones directas entre emisores y receptores, forzándolos a comunicarse indirectamente a través de un objeto mediador.
- *Observer* permite a los receptores suscribirse o darse de baja dinámicamente a la recepción de solicitudes.
- La diferencia entre **Mediator** y **Observer** a menudo resulta difusa. En la mayoría de los casos, puedes implementar uno de estos dos patrones; pero en ocasiones puedes aplicarlos ambos a la vez. Veamos cómo podemos hacerlo.

La meta principal del patrón *Mediator* consiste en eliminar las dependencias mutuas entre un grupo de componentes del sistema. En su lugar, estos componentes se vuelven dependientes de un único objeto mediador. La meta del patrón *Observer* es establecer conexiones dinámicas de un único sentido entre objetos, donde algunos objetos actúan como subordinados de otros.

Hay una implementación popular del patrón *Mediator* que se basa en el *Observer*. El objeto mediador juega el papel de notificador, y los componentes actúan como suscriptores que se suscriben o se dan de baja de los eventos del mediador. Cuando se implementa el *Mediator* de esta forma, puede asemejarse mucho al *Observer*.

Cuando te sientas confundido, recuerda que puedes implementar el patrón *Mediator* de otras maneras. Por ejemplo, puedes vincular permanentemente todos los componentes al mismo objeto mediador. Esta implementación no se parece al *Observer*, pero aún así será una instancia del patrón *Mediator*.

Ahora, imagina un programa en el que todos los componentes se hayan convertido en notificadores, permitiendo conexiones dinámicas entre sí. No hay un objeto mediador centralizado, tan solo un grupo distribuido de observadores.

Strategy

También llamado: Estrategia

Propósito

Strategy es un patrón de diseño de comportamiento que te permite definir una familia de algoritmos, colocar cada uno de ellos en una clase separada y hacer sus objetos intercambiables.



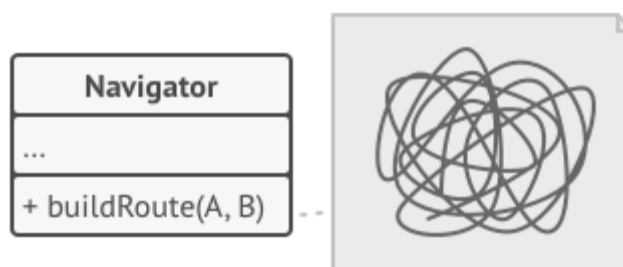
Problema

Un día decidiste crear una aplicación de navegación para viajeros ocasionales. La aplicación giraba alrededor de un bonito mapa que ayudaba a los usuarios a orientarse rápidamente en cualquier ciudad.

Una de las funciones más solicitadas para la aplicación era la planificación automática de rutas. Un usuario debía poder introducir una dirección y ver la ruta más rápida a ese destino mostrado en el mapa.

La primera versión de la aplicación sólo podía generar las rutas sobre carreteras. Las personas que viajaban en coche estaban locas de alegría. Pero, aparentemente, no a todo el mundo le gusta conducir durante sus vacaciones. De modo que, en la siguiente actualización, añadiste una opción para crear rutas a pie. Después, añadiste otra opción para permitir a las personas utilizar el transporte público en sus rutas.

Sin embargo, esto era sólo el principio. Más tarde planeaste añadir la generación de rutas para ciclistas, y más tarde, otra opción para trazar rutas por todas las atracciones turísticas de una ciudad.



El código del navegador se saturó.

Aunque desde una perspectiva comercial la aplicación era un éxito, la parte técnica provocaba muchos dolores de cabeza. Cada vez que añadías un nuevo algoritmo de enrutamiento, la clase principal del navegador doblaba su tamaño. En cierto momento, la bestia se volvió demasiado difícil de mantener.

Cualquier cambio en alguno de los algoritmos, ya fuera un sencillo arreglo de un error o un ligero ajuste de la representación de la calle, afectaba a toda la clase, aumentando las probabilidades de crear un error en un código ya funcional.

Además, el trabajo en equipo se volvió ineficiente. Tus compañeros, contratados tras el exitoso lanzamiento, se quejaban de que dedicaban demasiado tiempo a resolver conflictos de integración. Implementar una nueva función te exige cambiar la misma clase enorme, entrando en conflicto con el código producido por otras personas.

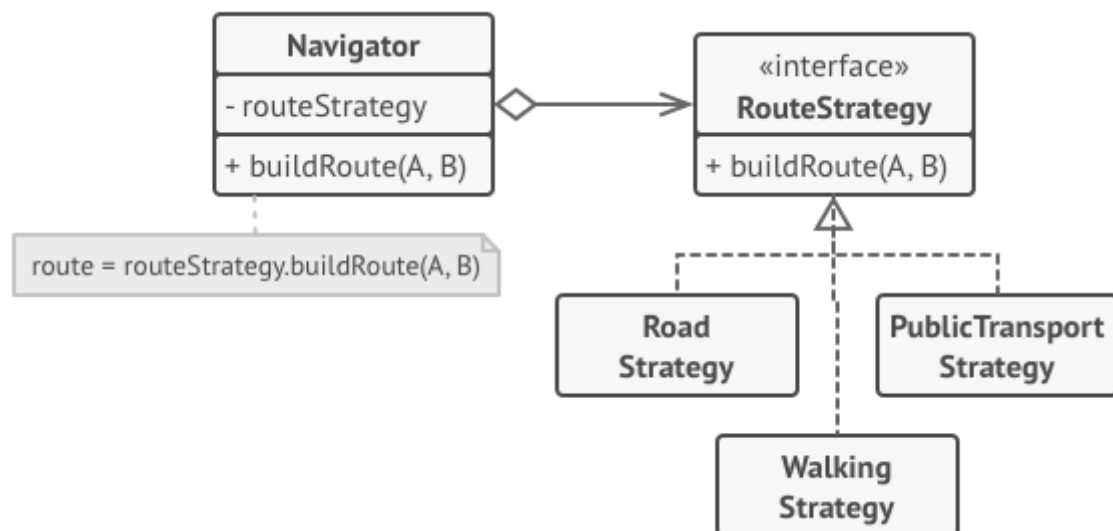
Solución

El patrón Strategy sugiere que tomes esa clase que hace algo específico de muchas formas diferentes y extraigas todos esos algoritmos para colocarlos en clases separadas llamadas *estrategias*.

La clase original, llamada *contexto*, debe tener un campo para almacenar una referencia a una de las estrategias. El contexto delega el trabajo a un objeto de estrategia vinculado en lugar de ejecutarlo por su cuenta.

La clase contexto no es responsable de seleccionar un algoritmo adecuado para la tarea. En lugar de eso, el cliente pasa la estrategia deseada a la clase contexto. De hecho, la clase contexto no sabe mucho acerca de las estrategias. Funciona con todas las estrategias a través de la misma interfaz genérica, que sólo expone un único método para disparar el algoritmo encapsulado dentro de la estrategia seleccionada.

De esta forma, el contexto se vuelve independiente de las estrategias concretas, así que puedes añadir nuevos algoritmos o modificar los existentes sin cambiar el código de la clase contexto o de otras estrategias.

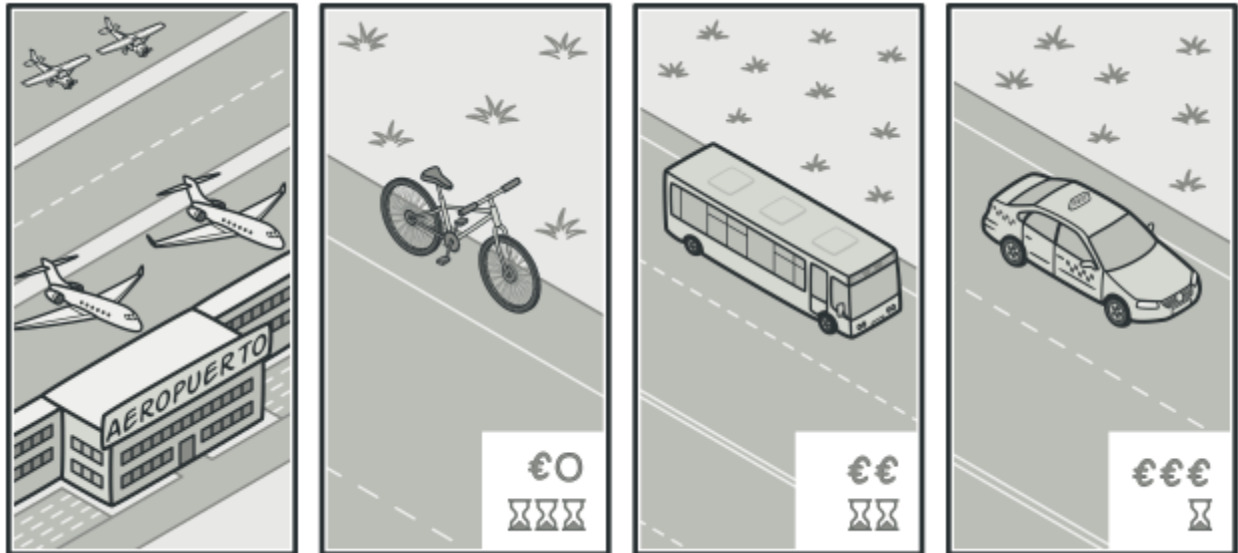


Estrategias de planificación de rutas.

En nuestra aplicación de navegación, cada algoritmo de enrutamiento puede extraerse y ponerse en su propia clase con un único método `crearRuta`. El método acepta un origen y un destino y devuelve una colección de puntos de control de la ruta.

Incluso contando con los mismos argumentos, cada clase de enrutamiento puede crear una ruta diferente. A la clase navegadora principal no le importa qué algoritmo se selecciona ya que su labor principal es representar un grupo de puntos de control en el mapa. La clase tiene un método para cambiar la estrategia activa de enrutamiento, de modo que sus clientes, como los botones en la interfaz de usuario, pueden sustituir el comportamiento seleccionado de enrutamiento por otro.

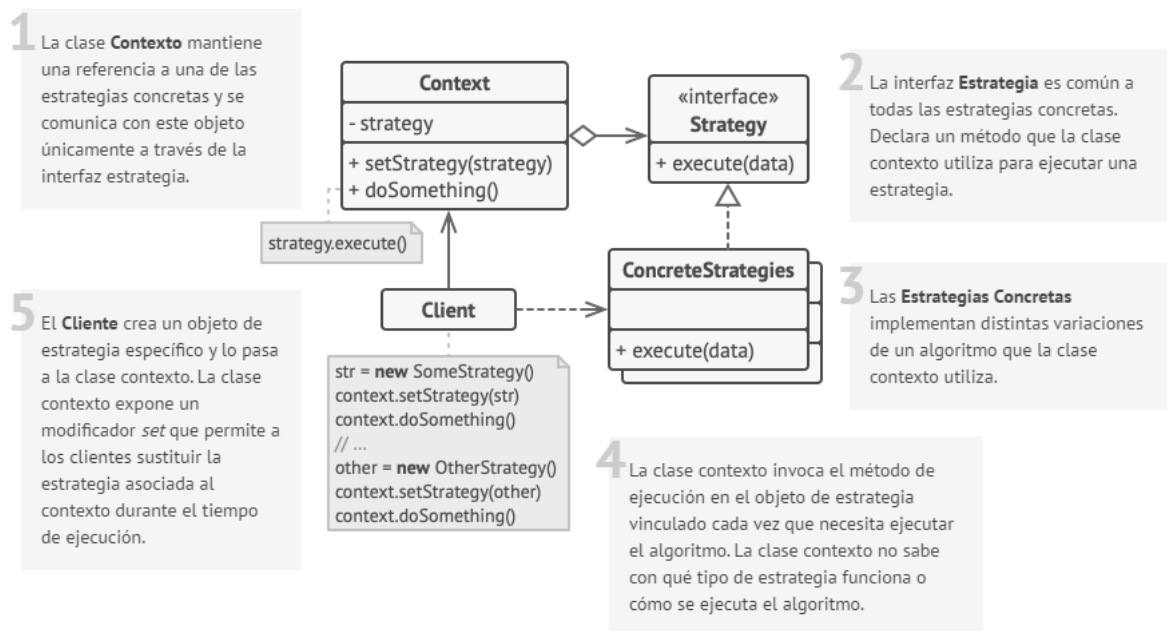
Analogía en el mundo real



Varias estrategias para llegar al aeropuerto.

Imagina que tienes que llegar al aeropuerto. Puedes tomar el autobús, pedir un taxi o ir en bicicleta. Éstas son tus estrategias de transporte. Puedes elegir una de las estrategias, dependiendo de factores como el presupuesto o los límites de tiempo.

Estructura



Pseudocódigo

En este ejemplo, el contexto utiliza varias **estrategias** para ejecutar diversas operaciones aritméticas.

```

// La interfaz estrategia declara operaciones comunes a todas
// las versiones soportadas de algún algoritmo. El contexto
// utiliza esta interfaz para invocar el algoritmo definido por
// las estrategias concretas.
interface Strategy is
    method execute(a, b)

// Las estrategias concretas implementan el algoritmo mientras
// siguen la interfaz estrategia base. La interfaz las hace
// intercambiables en el contexto.
class ConcreteStrategyAdd implements Strategy is
    method execute(a, b) is
        return a + b

class ConcreteStrategySubtract implements Strategy is
    method execute(a, b) is
        return a - b

class ConcreteStrategyMultiply implements Strategy is
    method execute(a, b) is
        return a * b

// El contexto define la interfaz de interés para los clientes.
class Context is
    // El contexto mantiene una referencia a uno de los objetos
    // de estrategia. El contexto no conoce la clase concreta de
    // una estrategia. Debe trabajar con todas las estrategias a
    // través de la interfaz estrategia.
    private strategy: Strategy

    // Normalmente, el contexto acepta una estrategia a través
    // del constructor y también proporciona un setter
    // (modificador) para poder cambiar la estrategia durante el
    // tiempo de ejecución.
    method setStrategy(Strategy strategy) is
        this.strategy = strategy

    // El contexto delega parte del trabajo al objeto de
    // estrategia en lugar de implementar varias versiones del
    // algoritmo por su cuenta.
    method executeStrategy(int a, int b) is
        return strategy.execute(a, b)

// El código cliente elige una estrategia concreta y la pasa al
// contexto. El cliente debe conocer las diferencias entre
// estrategias para elegir la mejor opción.
class ExampleApplication is
    method main() is
        Create context object.

        Read first number.
        Read last number.
        Read the desired action from user input.

        if (action == addition) then
            context.setStrategy(new ConcreteStrategyAdd())

        if (action == subtraction) then
            context.setStrategy(new ConcreteStrategySubtract())

        if (action == multiplication) then
            context.setStrategy(new ConcreteStrategyMultiply())

        result = context.executeStrategy(First number, Second
number)

        Print result.

```

Aplicabilidad

Utiliza el patrón Strategy cuando quieras utilizar distintas variantes de un algoritmo dentro de un objeto y poder cambiar de un algoritmo a otro durante el tiempo de ejecución.

El patrón Strategy te permite alterar indirectamente el comportamiento del objeto durante el tiempo de ejecución asociándolo con distintos subobjetos que pueden realizar subtarefas específicas de distintas maneras.

Utiliza el patrón Strategy cuando tengas muchas clases similares que sólo se diferencien en la forma en que ejecutan cierto comportamiento.

El patrón Strategy te permite extraer el comportamiento variante para ponerlo en una jerarquía de clases separada y combinar las clases originales en una, reduciendo con ello el código duplicado.

Utiliza el patrón para aislar la lógica de negocio de una clase, de los detalles de implementación de algoritmos que pueden no ser tan importantes en el contexto de esa lógica.

El patrón Strategy te permite aislar el código, los datos internos y las dependencias de varios algoritmos, del resto del código. Los diversos clientes obtienen una interfaz simple para ejecutar los algoritmos y cambiarlos durante el tiempo de ejecución.

Utiliza el patrón cuando tu clase tenga un enorme operador condicional que cambie entre distintas variantes del mismo algoritmo.

El patrón Strategy te permite suprimir dicho condicional extrayendo todos los algoritmos para ponerlos en clases separadas, las cuales implementan la misma interfaz. El objeto original delega la ejecución a uno de esos objetos, en lugar de implementar todas las variantes del algoritmo.

Implementación

1. En la clase contexto, identifica un algoritmo que tienda a sufrir cambios frecuentes. También puede ser un enorme condicional que seleccione y ejecute una variante del mismo algoritmo durante el tiempo de ejecución.
2. Declara la interfaz estrategia común a todas las variantes del algoritmo.
3. Uno a uno, extrae todos los algoritmos y ponlos en sus propias clases. Todas deben implementar la misma interfaz estrategia.
4. En la clase contexto, añade un campo para almacenar una referencia a un objeto de estrategia. Proporciona un modificador *set* para sustituir valores de ese campo. La clase contexto debe trabajar con el objeto de estrategia únicamente a través de la interfaz estrategia. La clase contexto puede definir una interfaz que permita a la estrategia acceder a sus datos.
5. Los clientes de la clase contexto deben asociarla con una estrategia adecuada que coincida con la forma en la que esperan que la clase contexto realice su trabajo principal.

Pros y contras

- ✓ Puedes intercambiar algoritmos usados dentro de un objeto durante el tiempo de ejecución.
- ✓ Puedes aislar los detalles de implementación de un algoritmo del código que lo utiliza.
- ✓ Puedes sustituir la herencia por composición.
- ✓ *Principio de abierto/cerrado*. Puedes introducir nuevas estrategias sin tener que cambiar el contexto.
- ✗ Si sólo tienes un par de algoritmos que raramente cambian, no hay una razón real para complicar el programa en exceso con nuevas clases e interfaces que vengan con el patrón.
- ✗ Los clientes deben conocer las diferencias entre estrategias para poder seleccionar la adecuada.
- ✗ Muchos lenguajes de programación modernos tienen un soporte de tipo funcional que te permite implementar distintas versiones de un algoritmo dentro de un grupo de funciones anónimas. Entonces puedes utilizar estas funciones exactamente como habrías utilizado los objetos de estrategia, pero sin saturar tu código con clases e interfaces adicionales.

Relaciones con otros patrones

- **Bridge**, **State**, **Strategy** (y, hasta cierto punto, **Adapter**) tienen estructuras muy similares. De hecho, todos estos patrones se basan en la composición, que consiste en delegar trabajo a otros objetos. Sin embargo, todos ellos solucionan problemas diferentes. Un patrón no es simplemente una receta para estructurar tu código de una forma específica. También puede comunicar a otros desarrolladores el problema que resuelve.
- **Command** y **Strategy** pueden resultar similares porque puedes usar ambos para parametrizar un objeto con cierta acción. No obstante, tienen propósitos muy diferentes.
 - Puedes utilizar *Command* para convertir cualquier operación en un objeto. Los parámetros de la operación se convierten en campos de ese objeto. La conversión te permite aplazar la ejecución de la operación, ponerla en cola, almacenar el historial de comandos, enviar comandos a servicios remotos, etc.
 - Por su parte, *Strategy* normalmente describe distintas formas de hacer lo mismo, permitiéndote intercambiar estos algoritmos dentro de una única clase contexto.
- **Decorator** te permite cambiar la piel de un objeto, mientras que **Strategy** te permite cambiar sus entrañas.
- **Template Method** se basa en la herencia: te permite alterar partes de un algoritmo extendiendo esas partes en subclases. **Strategy** se basa en la composición: puedes alterar partes del comportamiento del objeto suministrándole distintas estrategias que se correspondan con ese comportamiento. *Template Method* trabaja al nivel de la clase, por lo que es estático. *Strategy* trabaja al nivel del objeto, permitiéndote cambiar los comportamientos durante el tiempo de ejecución.
- **State** puede considerarse una extensión de **Strategy**. Ambos patrones se basan en la composición: cambian el comportamiento del contexto delegando parte del trabajo a objetos ayudantes. *Strategy* hace que estos objetos sean completamente independientes y no se conozcan entre sí. Sin embargo, *State* no restringe las dependencias entre estados concretos, permitiéndoles alterar el estado del contexto a voluntad.

Tabla de contenido

¿Qué es un patrón de diseño?	1
¿En qué consiste el patrón?	1
Historia de los patrones	1
Ventaja de los patrones	2
¿Por qué debería aprender sobre patrones?	2
Crítica de los patrones	2
Clasificación de patrones	3
Por su nivel.....	3
Por su propósito.....	3
Patrones creacionales	3
Patrones estructurales	3
Patrones de comportamiento	3
El catálogo de patrones de diseño	4
1. Patrones creacionales:	4
2. Patrones estructurales:	4
3. Patrones de comportamiento:	5
PATRONES CREACIONALES	5
Factory Method.....	5
Propósito	5
Estructura.....	7
Pseudocódigo.....	8
Aplicabilidad.....	10
Implementación	11
Pros y contras.....	11
Relaciones con otros patrones.....	12
Abstract Factory	13
Propósito	13
Estructura.....	16
Pseudocódigo.....	16
Aplicabilidad.....	19
Implementación	19
Pros y contras.....	19
Relaciones con otros patrones.....	19
Singleton	21
Propósito	21
Estructura.....	22
Pseudocódigo.....	22
Aplicabilidad.....	24

Implementación	24
Pros y contras.....	24
Relaciones con otros patrones.....	24
PATRONES ESTRUCTURALES	27
Adapter	27
Propósito.....	27
Problema	27
Solución.....	27
Analogía en el mundo real	28
Estructura.....	29
Pseudocódigo	30
Aplicabilidad.....	32
Implementación	32
Pros y contras.....	32
Relaciones con otros patrones.....	32
Composite	34
Propósito.....	34
Problema	34
Solución.....	35
Analogía en el mundo real	35
Estructura.....	36
Pseudocódigo	36
Aplicabilidad.....	39
Implementación	39
Pros y contras.....	39
Relaciones con otros patrones.....	40
Facade	41
Propósito.....	41
Problema	41
Solución.....	41
Analogía en el mundo real	42
Estructura.....	42
Pseudocódigo	43
Aplicabilidad.....	45
Implementación	45
Pros y contras.....	45
Relaciones con otros patrones.....	45
PATRONES DE COMPORTAMIENTO	47
Observer.....	47

Propósito	47
Problema	47
Solución	48
Analogía en el mundo real	49
Estructura	50
Pseudocódigo	50
Aplicabilidad	52
Implementación	52
Pros y contras	52
Relaciones con otros patrones	52
Strategy	54
Propósito	54
Problema	54
Solución	55
Analogía en el mundo real	56
Estructura	56
Pseudocódigo	56
Aplicabilidad	58
Implementación	58
Pros y contras	58
Relaciones con otros patrones	59